

Module 11 - chapter 6

Deploy to EKS Cluster from Jenkins Pipeline

The commands used in this lecture and the config.yaml file can be found here: <https://gitlab.com/twn-devops-bootcamp/latest/11-eks/deploy-to-eks-from-jenkins>

The java-maven-project used can be found here: <https://gitlab.com/twn-devops-bootcamp/latest/11-eks/java-maven-app>



I deleted the eps cluster from Module 11 -chapter 5

To delete the cluster created with eksctl

```
-> eksctl delete cluster --name demo-cluster
```

eksctl automatically cleans up kubeconfig — it removes the context, cluster and user entries for the deleted cluster.

So I had to create it again:

```
->
eksctl create cluster \
--name demo-cluster \
--version 1.35 \
--region eu-west-2 \
--nodegroup-name demo-nodes \
--node-type t2.micro \
--nodes 2 \
--nodes-min 1 \
--nodes-max 3
```

And everything is working as expected:

```

2026-06-09 09:37:25 [✓] all EKS cluster resources for "demo-cluster" have been created
2026-06-09 09:37:25 [i] nodegroup "demo-nodes" has 2 node(s)
2026-06-09 09:37:25 [i] node "ip-192-168-60-249.eu-west-2.compute.internal" is ready
2026-06-09 09:37:25 [i] node "ip-192-168-65-77.eu-west-2.compute.internal" is ready
2026-06-09 09:37:25 [i] waiting for at least 1 node(s) to become ready in "demo-nodes"
2026-06-09 09:37:25 [i] nodegroup "demo-nodes" has 2 node(s)
2026-06-09 09:37:25 [i] node "ip-192-168-60-249.eu-west-2.compute.internal" is ready
2026-06-09 09:37:25 [i] node "ip-192-168-65-77.eu-west-2.compute.internal" is ready
2026-06-09 09:37:25 [✓] created 1 managed nodegroup(s) in cluster "demo-cluster"
2026-06-09 09:37:26 [i] creating addon: metrics-server
2026-06-09 09:37:26 [i] successfully created addon: metrics-server
2026-06-09 09:37:26 [i] kubectl command should work with "/Users/astrid/.kube/config", try 'kubectl get nodes'
2026-06-09 09:37:26 [✓] EKS cluster "demo-cluster" in "eu-west-2" region is ready
astrid@192.168.1.112 /Users/astrid
% kubectl get node
NAME                                     STATUS    ROLES    AGE    VERSION
ip-192-168-60-249.eu-west-2.compute.internal Ready    <none>    3m16s  v1.35.5-eks-3385e9b
ip-192-168-65-77.eu-west-2.compute.internal Ready    <none>    3m17s  v1.35.5-eks-3385e9b
astrid@192.168.1.112 /Users/astrid
% kubectl get pod
No resources found in default namespace.

```

And the kubeconfig file has two contexts (an old one created when I used AWS CLI and the context for the eksctl cluster that I created today, the one created in chapter 5 was cleaned up in this file when I deleted the old eksctl cluster)

->cat .kube/config

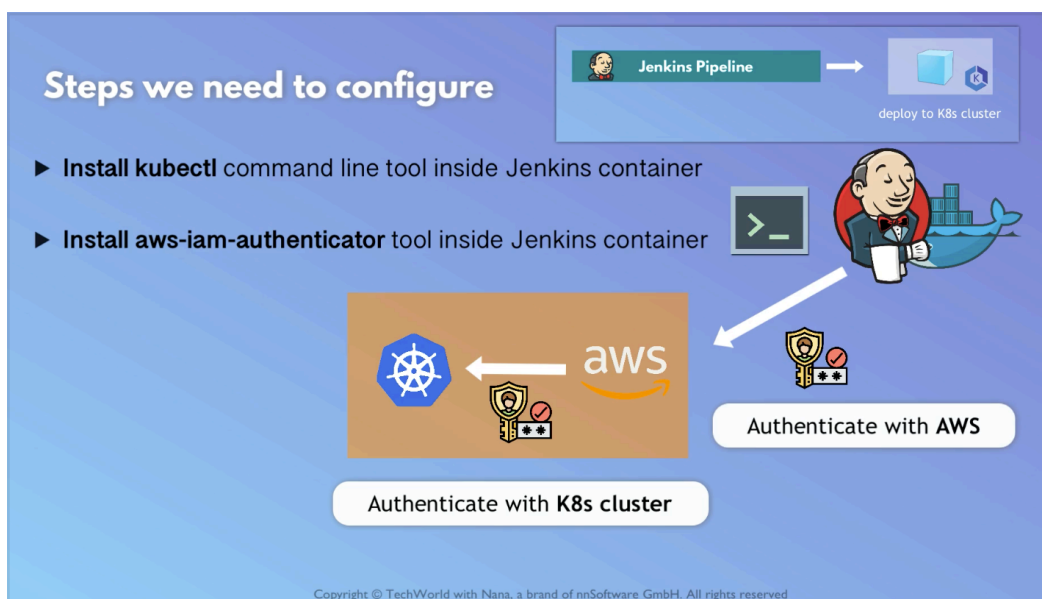
```
% cat .kube/config
apiVersion: v1
clusters:
- cluster:
  certificate-authority-data: LS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0tLS0tCk1JSURCVENDQWUyZ0F
  TkRSYUZ3MHP0akExTWpReE16UTJ0RFJhTUJJVeApFekFSQmdOVkJBTVRDbXQxWW1WeWJtVjBaWE13Z2dFaU1BMEd
  SVpnmGoKk51T0p3eVAzdGVrdWYrVUw2ZmN4WUtFVGJ0bzR1TXJmYnhaNzNDZG1QWE4rMzFBN3grWUFwaHJrbVh
  WWVnb092WVI0MkVURDdmVXhrc1lBbDFjWlNsS1lSL1ZTQkNJUmRGSmR2WlZTb1kKYVhVRjF4RWpjcFpQV0t6UW5
  TUE0R0ExVWREd0VCL3dRRUF3SUNwREFQckJnTlZlUk1CQWY4RUJUQURBUUgvTUIwR0ExVWREZ1FXQkJTTnV6c1p
  UwpPZkgrN3kwYjd0Z3VHem0xdTZak5XZ1hVcmhzbHVkb3BSWHdHY0t2VndvOUZDVmhVdTdNUVEwS1ZkZDBzcVg
  Q3lBYVU4TUUpkWXhrcUs1Yk1QRmIzYTNPszJtc2xLOFlVZWWhGUepiS1poawpBRU43RnNxY2lyLzNSaEpHSzRNBdd
  ekhiTnJrNkVmc55d2dDV0RMVmqKbUVQcWE5MVQ1Vm1jCi0tLS0tRU5EIENFU1RJRklDQVRFLS0tLS0K
  server: https://2935C95E1D96128C237B75A7340230B3.gr7.eu-west-2.eks.amazonaws.com
  name: arn:aws:eks:eu-west-2:706976333940:cluster/eks-cluster-test
- cluster:
  certificate-authority-data: LS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0tLS0tCk1JSURCVENDQWUyZ0F
  TkRWYUZ3MHP0akEyTURZd09ESTV0RFZhTUJJVeApFekFSQmdOVkJBTVRDbXQxWW1WeWJtVjBaWE13Z2dFaU1BMEd
  cUUyQ0skTU51VUR0aG1BaFlScTVsZnU2YzZGQW1sNWNncEVnL3pFZ1RSNzFKTkFFNXpPWXFkVzljZjJlJRG15ckx
  RzRWUThJRDAxWVhDR2cvdVY1SWdQL3p6WGQwQmFqSkJ5QXNXUW5IR3pDaklSWFyYkY1gzeXVWZHZOb1FCNUhXTGt
  TUE0R0ExVWREd0VCL3dRRUF3SUNwREFQckJnTlZlUk1CQWY4RUJUQURBUUgvTUIwR0ExVWREZ1FXQkJRREXUNnp
  NAppVURhSTMvOEZkcWtTNkN2azhpZmU2R0VjejZVbGNhbEpUbjBLaTRmOXVXWE5GREU1c1lyRlIwbXN2YmVHQXh
  MkRHdnJybKvJmJIM3JtRExpWfK1Wm1QNTNVXlVUZVWRnBTUnBPSmFWbgpFVTlnbm02ZHJuR2doZVFBTKVORGX
  MmsrcG1HSHNVMGfucmZvS3hXNEgKa1RzSTNMCHRRM0txCi0tLS0tRU5EIENFU1RJRklDQVRFLS0tLS0K
  server: https://EBFD13B05DA0171EA96228C7F6E79CFD.y14.eu-west-2.eks.amazonaws.com
  name: demo-cluster.eu-west-2.eksctl.io
contexts:
- context:
  cluster: demo-cluster.eu-west-2.eksctl.io
  user: admin@demo-cluster.eu-west-2.eksctl.io
  name: admin@demo-cluster.eu-west-2.eksctl.io
- context:
  cluster: arn:aws:eks:eu-west-2:706976333940:cluster/eks-cluster-test
  user: arn:aws:eks:eu-west-2:706976333940:cluster/eks-cluster-test
  name: arn:aws:eks:eu-west-2:706976333940:cluster/eks-cluster-test
current-context: admin@demo-cluster.eu-west-2.eksctl.io
kind: Config
users:
- name: admin@demo-cluster.eu-west-2.eksctl.io
  user:
    exec:
      apiVersion: client.authentication.k8s.io/v1beta1
      args:
      - token
      - -i
      - demo-cluster
      command: aws-iam-authenticator
```

The cluster is fresh, we have nodes but we don't have application pods running yet.



Steps we need to configure:

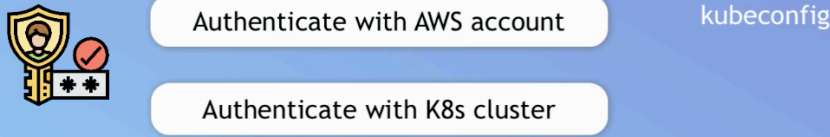
1. Install **kubectl** command line tool inside Jenkins container -> so Jenkins can use kubectl
2. Install **aws-iam-authenticator** tool inside Jenkins container -> When Jenkins tries to connect to the EKS cluster it will need to authenticate with:
 - EKS and
 - AWS



3. Install Kubeconfig file to connect to EKS cluster -> it will contain Information to:
 - Authenticate with AWS account (using the aws-iam-authenticator command line tool)
 - Authenticate with k8s cluster - EKS

► **Create kubeconfig** file to connect to EKS cluster

► Contains all the necessary information for authentication



Authenticate with AWS account

Authenticate with K8s cluster

kubeconfig

Copyright © TechWorld with Nana, a brand of nnSoftware GmbH. All rights reserved

In our local machines we downloaded a config file which configured kubectl to talk to the EKS cluster

```
nana@macbook /Users/nana
% ls .kube/config
```



locally

.kube/config file

aws

Note: when we installed eksctl in my local machine, aws-iam-authenticator got automatically installed locally as well.

So locally I have the kubeconfig file and aws-iam-authenticator:
-> aws-iam-authenticator help

```

astrid@192.168.1.112 /Users/astrid
% aws-iam-authenticator help
A tool to authenticate to Kubernetes using AWS IAM credentials

Usage:
  aws-iam-authenticator [command]

Available Commands:
  add          add IAM entity to an existing aws-auth configmap
  completion  Generate the autocompletion script for the specified shell
  help        Help about any command
  init        Pre-generate certificate, private key, and kubeconfig files for the server.
  server      Run a webhook validation server suitable that validates tokens using AWS IAM
  token       Authenticate using AWS IAM and get token for Kubernetes
  verify      Verify a token for debugging purpose
  version     Version will output the current build information

Flags:
  -i, --cluster-id ID          Specify the cluster ID, a unique-per-cluster identifier for your aws-iam-authenticator installation.
  -c, --config filename       Load configuration from filename
  --feature-gates mapStringBool A set of key=value pairs that describe feature gates for alpha/experimental features. Options are:
                               AllAlpha=true|false (ALPHA - default=false)
                               AllBeta=true|false (BETA - default=false)
                               ConfiguredInitDirectories=true|false (ALPHA - default=false)
                               IAMIdentityMappingCRD=true|false (ALPHA - default=false)
                               SSORoleMatch=true|false (ALPHA - default=false)
  -h, --help                  help for aws-iam-authenticator
  -l, --log-format string     Specify log format to use when logging to stderr [text or json] (default "text")
  -v, --log-verbosity string  Specify log level to use when logging to stderr [panic, fatal, error, warning, info, debug, trace] (default "info")

Use "aws-iam-authenticator [command] --help" for more information about a command.
astrid@192.168.1.112 /Users/astrid
%

```

-> ls .kube

```

astrid@192.168.1.112 /Users/astrid
% ls .kube
cache  config  kubens
astrid@192.168.1.112 /Users/astrid
%

```

The combination of kubconfig file and aws-iam-authenticator allows me to execute kubectl locally and connect to EKS cluster and get a response when I run the below for example:

-> kubectl get node

And we need Jenkins to be able to do the same. But for Jenkins we will create the kubeconfig file manually.

4. Add AWS credentials on Jenkins for AWS account authentication: so we can authenticate with AWS account, we did this locally by executing

-> aws configure

But for Jenkins it can't be hardcoded.

From Module 9 - chapter 3 and chapter 12:

Retrieve access keys Info

Access key
If you lose or forget your secret access key, you cannot retrieve it. Instead, create a new access key and make the old key inactive.

Access key	Secret access key
 AKIAUZ7NN5BBFSZ6YL46	 ***** Show

Access key best practices

- Never store your access key in plain text, in a code repository, or in code.
- Disable or delete access key when no longer needed.
- Enable least-privilege permissions.
- Rotate access keys regularly.

For more details about managing access keys, see the [Best practices for managing AWS access keys](#).

[Download .csv file](#) [Done](#)

5. Adjust Jenkinsfile to configure EKS cluster deployment

Steps we need to configure

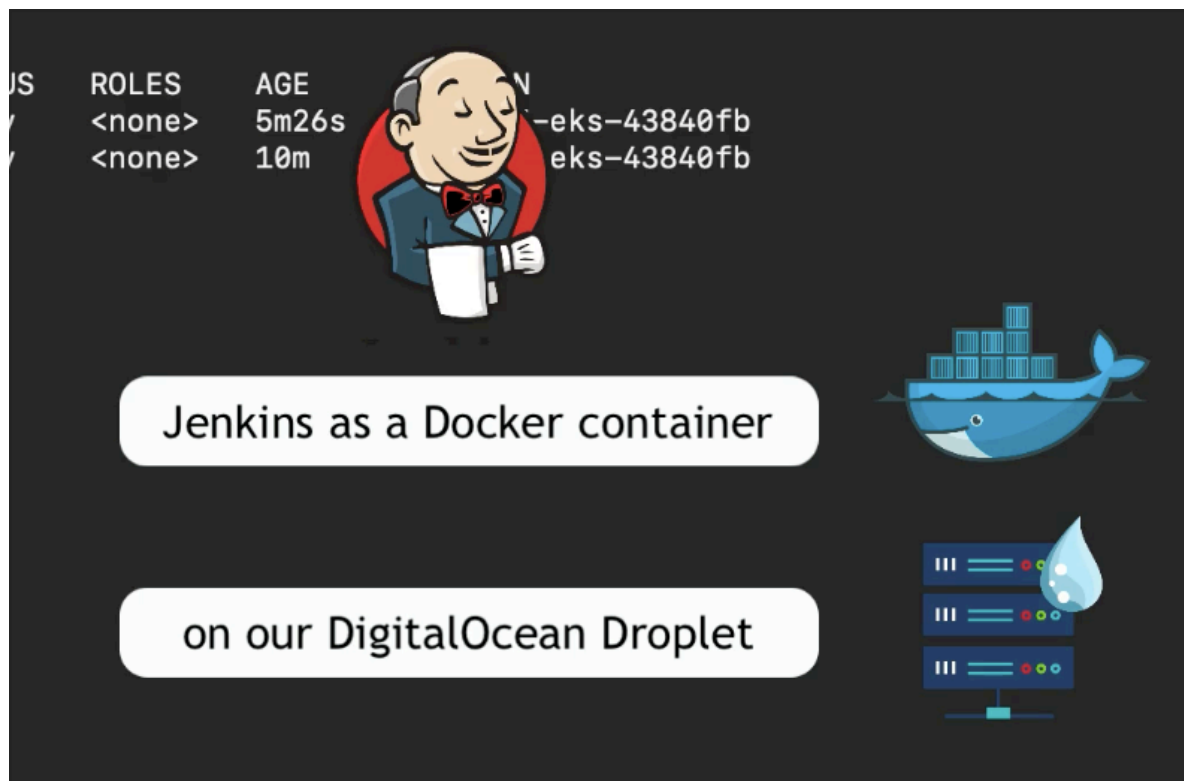
 Jenkins Pipeline →  deploy to K8s cluster

- ▶ **Install kubectl** command line tool inside Jenkins container
- ▶ **Install aws-iam-authenticator** tool inside Jenkins container
- ▶ **Create kubeconfig** file to connect to EKS cluster
- ▶ **Add AWS credentials** on Jenkins for AWS account authentication
- ▶ **Adjust Jenkinsfile** to configure EKS cluster deployment

Copyright © TechWorld with Nana, a brand of nnSoftware GmbH. All rights reserved

Let's start:

1. Install kubectl on Jenkins Server



Droplet is running and Jenkins container is running:

Build Queue				
Build Executor Status (0 of 2 executors busy)				
S	W	Name ↓	Last Success	Last Failure
		aws-multibranch-pipeline	1 mo 15 days log	N/A
		java-maven-build	6 mo 15 days #6	2 mo 22 days
		microservices-deploy	17 days #4	17 days #
		my-freestyle-job	6 mo 18 days #3	N/A
		my-multi-branch-pipeline	2 mo 17 days log	N/A
		my-pipeline	2 mo 17 days #194	2 mo 18 days

I ssh into Jenkins:

-> ssh **root@165.22.116.21**

And then check the running containers:

-> docker ps

```

astrid@192.168.1.112 /Users/astrid
% ssh root@165.22.116.21
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-71-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Tue Jun  9 09:56:33 UTC 2026

System load:  0.05          Processes:      125
Usage of /:   14.2% of 76.45GB Users logged in: 0
Memory usage: 32%          IPv4 address for eth0: 165.22.116.21
Swap usage:   0%           IPv4 address for eth0: 10.16.0.6

Expanded Security Maintenance for Applications is not enabled.

65 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

1 additional security update can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

*** System restart required ***
Last login: Sat Jun  6 13:33:13 2026 from 140.228.47.252
root@jenkins-server:~# docker ps

```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
655647e3fb2e	jenkins/jenkins:lts	"/usr/bin/tini -- /u..."	2 weeks ago	Up 2 weeks	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp

```

root@jenkins-server:~#

```

Jenkins user doesn't have permissions to install so we need to use root user instead. So we access Jenkins container as root user

-> docker exec -u 0 -it 655647e3fb2e bash

```

docker exec -u 0          # user ID 0 = root
docker exec -u root      # same, using name instead of ID
docker exec              # no -u flag = uses whatever user the container runs
as by default

```

```

root@jenkins-server:~# docker ps

```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
655647e3fb2e	jenkins/jenkins:lts	"/usr/bin/tini -- /u..."	2 weeks ago	Up 2 weeks	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp

```

root@jenkins-server:~# docker exec -u 0 -it 655647e3fb2e bash
root@655647e3fb2e:/#

```

Now we will download kubectl with the latest stable version:

-> curl -LO https://storage.googleapis.com/kubernetes-release/release/\$(curl -s https://storage.googleapis.com/kubernetes-release/release/stable.txt)/bin/linux/amd64/kubectl

Change mode of the kubectl file to be executable

-> chmod +x ./kubectl

Move kubectl executable file:

-> mv ./kubectl /usr/local/bin/kubectl

I can run the three commands like this as well:

-> curl -LO https://storage.googleapis.com/kubernetes-release/release/\$(curl -s https://storage.googleapis.com/kubernetes-release/release/stable.txt)/bin/linux/amd64/kubectl; chmod +x ./kubectl; mv ./kubectl /usr/local/bin/kubectl

But I am doing it one by one:

```

root@jenkins-server:~# docker exec -it 655647e3fb2e bash
root@655647e3fb2e:/# curl -LO https://storage.googleapis.com/kubernetes-release/release/$(curl -s https://storage.googleapis.com/kubernetes-release/release/stable.txt)/bin/linux/
amd64/kubectl
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100 53.7M  100 53.7M    0     0  34.6M    0  0:00:01  0:00:01 --:--:-- 34.6M
root@655647e3fb2e:/#

```

```

root@655647e3fb2e:/# ls
LICENSE  README-zh_CN.md  README.md  bin  boot  dev  etc  helmfile.tar.gz  home  kubectl  lib
root@655647e3fb2e:/# ls -la kubectl
-rw-r--r-- 1 root root 56381592 Jun  9 10:14 kubectl
root@655647e3fb2e:/#

```

Change mode of the kubectl file to be executable
-> `chmod +x ./kubectl`

```

root@655647e3fb2e:/# chmod +x ./kubectl
root@655647e3fb2e:/# ls -la kubectl
-rwxr-xr-x 1 root root 56381592 Jun  9 10:14 kubectl
root@655647e3fb2e:/#

```

Move kubectl executable file:
-> `mv ./kubectl /usr/local/bin/kubectl`

`/usr/local/bin/kubectl` #make sure the name of the file is 'kubectl'

kubectl is already in my Jenkins folder `/usr/local/bin/kubectl` (I installed it when playing with Jenkins after finishing Module 10 - chapter 24) and it has the executable permissions.

So I don't need to move the file and replace it. I will keep the one I already have in `/usr/local/bin` and remove the one I just downloaded
-> `rm ./kubectl`

Now let's check
-> `kubectl version`

```

root@655647e3fb2e:/# kubectl version
Client Version: v1.36.1
Kustomize Version: v5.8.1
Error from server (Forbidden): <html><head><meta http-equiv='refresh' content=
ipt id='redirect' data-redirect-url='/login?from=%2Fversion%3Ftimeout%3D32s'
/head><body style='background-color:white; color:white;'>
Authentication required
<!--
-->

</body></html>
root@655647e3fb2e:/# █

```

Because we are not connected to any cluster we get 'authentication required'

2. Install AWS IAM authenticator

Specific to AWS EKS

```

-> curl -Lo aws-iam-authenticator https://github.com/kubernetes-sigs/aws-
iam-authenticator/releases/download/v0.6.11/aws-iam-
authenticator_0.6.11_linux_amd64
-> chmod +x ./aws-iam-authenticator
-> mv ./aws-iam-authenticator /usr/local/bin

```

```

root@655647e3fb2e:/# curl -Lo aws-iam-authenticator https://github.com/kubernetes-sigs/aws-iam-authenticator/releases/download/v0.6.11/aws-iam-authenticator_0.6.11_linux_amd64
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left     Speed
  0     0    0     0     0     0      0      0  --:--:-- --:--:-- --:--:--    0
100 51.0M 100 51.0M    0     0 30.3M      0  0:00:01  0:00:01 --:--:-- 34.9M
root@655647e3fb2e:/# ls
LICENSE      README.md    bin  dev  helmfile.tar.gz  lib  media  opt  root  sbin  sys  usr
README-zh_CN.md  aws-iam-authenticator  boot  etc  home          lib64  mnt  proc  run  srv  tmp  var
root@655647e3fb2e:/# █

```

```

root@655647e3fb2e:/# ls -la ./aws-iam-authenticator
-rw-r--r-- 1 root root 53522432 Jun  9 10:42 ./aws-iam-authenticator
root@655647e3fb2e:/# chmod +x ./aws-iam-authenticator
root@655647e3fb2e:/# ls -la ./aws-iam-authenticator
-rwxr-xr-x 1 root root 53522432 Jun  9 10:42 ./aws-iam-authenticator
root@655647e3fb2e:/# █

```

```

root@655647e3fb2e:/# mv ./aws-iam-authenticator /usr/local/bin/
root@655647e3fb2e:/# ls /usr/local/bin/
aws-iam-authenticator  git-lfs  helm  helmfile  jenkins-support  jenkins.sh  kubectl
root@655647e3fb2e:/# █

```

```

root@655647e3fb2e:/# /usr/local/bin/aws-iam-authenticator help
A tool to authenticate to Kubernetes using AWS IAM credentials

Usage:
  aws-iam-authenticator [command]

Available Commands:
  add          add IAM entity to an existing aws-auth configmap
  completion   Generate the autocompletion script for the specified shell
  help         Help about any command
  init         Pre-generate certificate, private key, and kubeconfig files for the server.
  server       Run a webhook validation server suitable that validates tokens using AWS IAM
  token        Authenticate using AWS IAM and get token for Kubernetes
  verify       Verify a token for debugging purpose
  version      Version will output the current build information

Flags:
  -i, --cluster-id ID          Specify the cluster ID, a unique-per-cluster identifier for your
                             llation.
  -c, --config filename       Load configuration from filename
  --feature-gates mapStringBool A set of key=value pairs that describe feature gates for alpha/ex
                             s are:
                             AllAlpha=truelfalse (ALPHA - default=false)
                             AllBeta=truelfalse (BETA - default=false)
                             ConfiguredInitDirectories=truelfalse (ALPHA - default=false)
                             IAMIdentityMappingCRD=truelfalse (ALPHA - default=false)
  -h, --help                  help for aws-iam-authenticator
  -l, --log-format string      Specify log format to use when logging to stderr [text or json]

Use "aws-iam-authenticator [command] --help" for more information about a command.

```

- ✓ ▶ **Install kubectl** command line tool inside Jenkins container
- ✓ ▶ **Install aws-iam-authenticator** tool inside Jenkins container
- ▶ **Create kubeconfig** file to connect to EKS cluster
- ▶ **Add AWS credentials** on Jenkins for AWS account authentication
- ▶ **Adjust Jenkinsfile** to configure EKS cluster deployment

3. Create Kubeconfig file for Jenkins -> manually

Jenkins container doesn't have an editor because is lightweight, so we exit the container and use the droplet instead:

-> exit

```

root@655647e3fb2e:/# exit
exit
root@jenkins-server:~#

```

Now let's create the config file manually:

-> vim config


```
apiVersion: v1
kind: Config
clusters:
- cluster:
    certificate-authority-data: <certificate-data>
    server: <endpoint-url>
    name: kubernetes
contexts:
- context:
    cluster: kubernetes
    user: aws
    name: aws
current-context: aws
users:
- name: aws
  user:
    exec:
      apiVersion: client.authentication.k8s.io/v1beta1
      command: /usr/local/bin/aws-iam-authenticator
      args:
        - "token"
        - "-i"
        - <cluster-name>
```

```
~
~
~
~
~
~
~
~
~
```

```
:wq
```

I need to update some of the text in the template above 🙏

Values we need to update:

1) K8s cluster name

2) Server Endpoint

3) certificate-authority-data

- <cluster-name> -> "demo-cluster"
- <endpoint-url> -> <https://EBFD13B05DA0171EA96228C7F6E79CFD.yl4.eu-west-2.eks.amazonaws.com>

Public endpoint for talking to the cluster

-> <https://EBFD13B05DA0171EA96228C7F6E79CFD.yl4.eu-west-2.eks.amazonaws.com>

Amazon Elastic Kubernetes Service > Clusters > demo-cluster

Amazon Elastic Kubernetes Service

Dashboard
Clusters

▼ **Settings**
Dashboard settings
Console settings

▼ **Amazon EKS Anywhere**
Enterprise Subscriptions

▼ **Related services**
Amazon ECR [↗](#)
AWS Batch [↗](#)

Documentation [↗](#)
EKS workshops [↗](#)

demo-cluster

ⓘ A new Kubernetes version is available for this cluster.

ⓘ ▶ End of standard support for Kubernetes version 1.35 is March 27, 2027.

▼ **Cluster info** [Info](#)

Status ✓ Active	Kubernetes version 1.35
Cluster health ✓ 0	Upgrade insights ✓ 5

[Overview](#) | [Resources](#) | [Compute](#) | [Networking](#) | [Add-on](#)

Details

API server endpoint
<https://EBFD13B05DA0171EA96228C7F6E79CFD.yl4.eu-west-2.eks.amazonaws.com>

Certificate authority
 LS0tLS1CRUdJTiBDRVJUSUZJQ0FUR50tLS0tCk1JSURCVENDQWUyZ0F3SUJBZ0lUYXZra1dQTGV0Q0V3RFFZSktvWklodmNOQVFFTEJRQXdGVEVUTUJFR0ExVUUKQXhNS2EzVmlaWEp1WlhSbGN6

- <certificate-data> ->

We get it from our local kubeconfig file, so I need to change the terminal to check my local file:

```
astrid@192.168.1.112 /Users/astrid
% cat .kube/config
```

-> cat .kube/config

I look for 'certificate-authority-data' and I have two, the one I want is the want for cluster demo-cluster which is the second one:

```

astrid@192.168.1.112 /Users/astroid
% cat .kube/config

apiVersion: v1
clusters:
- cluster:
    certificate-authority-data: LS0tLS1CRUdJTiBDRVJSU2UzJ0FURS0tLS0tCk1JSURCVENDQWUyZ0F3SUJBZ0J1JGVI5bkiYtTndHA1V3RFRZJzU1UjRFR0ExVUUKQXhNS2EzZm1tZWp1IWI1bSbGN6QGdzB5TmptBMU1Yq3hNe1F4TKRSYU3Mhpo0aEYtWPRe16UTJ3ORFJhTJUJVeApFekFSQmd0VkJhTBRVJ1BMEdDU3FHU0t1iM0RRRUJBUVVBQTRJQkR3QXdnZ0VLCkFvSUJBUURHTM1XNDzbW5VbW9tQU1LD0Z5Nk5jdndkeDmrTVRRb0NkbGxqdEI4VGJFdXp1NFtF1ScTSvZm1Y2ZzQWw1SNVNNcEnH13pF21RSN5ZFKTKrFFNXpPWXfKvZ1jdj1JRG15ckxxxZn1OWApSU1BVFhFSZ0Z0qj1QVZ2JZW5LczBhL2xGb0F5eW1uZU1CRBtGNG1XU1JmQng1cjY3Zm1lc4UcpWb0k0rMFA5R2RWUthJRDAXVvHdR2cdvY15WdQ13p6WGQwQmFqSk53QXNXUW5IR3pDak1SWFYKY1gzeXVWZH20LazHBnd1NNTDhNvj5da09TMeJdVGY3bE0yb000M1lyZVNMDApwQNdVSVJRbVfScVh3aFRbjTRFOULTQzhzNUZiQWdNQkFBR2pXVEJYTUE0R0ExVWREd0V1C1QWY4R0U1JGUURBUlUvGtUIwR0ExVWREZ1F0XkQJRRcUnzppWmmFMYnFslZrVNUdNmBIZH0UUFUFRYQYQmd0VkhSRUVEakFNZ2dwcMkRXSmxjbtVSzEdWek1TRQJYFR0UjPWRNRkZa0ZdNAppVURHSTMV0EzKwZbWtNkNzaxzhZm2R0VfvejZvGNbHEpUbjJhY0TRM0XVXW5GREU1c1lyI1wbNzYmVHQXh5C11uV0UdXk1OW9pGkKv0REXeYrF5Ym1lYmdS0TlmrXpSNz1PRGxUzINWqG5xdm0kYjRYdFZWVcsvg2TJRnREzNux1d0dWmkRhDnjYkYvJmJIM3j2RExpWfK1Wm1QNTNVMnbm0Z2HJZdo2VFBTKvORGXNdDhCER3a2hU0UJzdX1naGNIb3FOVKNbTMMNRkjJTQnFeXGsuSnpTbJx3Ck1ZcnBINXRI1FUNXW0E10b21Ta0drT2tsrcG1HSHNVMGfucm253NhEgKa1RSTNMCHRRM0txC0tLS0tRUSEIENFUR1Rk1DQVRFLS0tLS0K

server: https://2935C95E1D96128C237B75A7340230B3.gr7.eu-west-2.eks.amazonaws.com
name: arn:aws:eks:eu-west-2:706976333940:eks-us-east-cluster-test
- cluster:
    certificate-authority-data: LS0tLS1CRUdJTiBDRVJSU2UzJ0FURS0tLS0tCk1JSURCVENDQWUyZ0F3SUJBZ0J1JYXZra1dQTVG0QV03RFRZJzU1UjRFR0ExVUUKQXhNS2EzZm1tZWp1IWI1bSbGN6QGdzB5TmptBMk1Ea3dPREkwTKRWYU3Mhpo0aEYtUR2d09ESTVORFZhtJUJVeApFekFSQmd0VkJhTBRVJ1BMEdDU3FHU0t1iM0RRRUJBUVVBQTRJQkR3QXdnZ0VLCkFvSUJBUURHTM1XNDzbW5VbW9tQU1LD0Z5Nk5jdndkeDmrTVRRb0NkbGxqdEI4VGJFdXp1NFtF1ScTSvZm1Y2ZzQWw1SNVNNcEnH13pF21RSN5ZFKTKrFFNXpPWXfKvZ1jdj1JRG15ckxxxZn1OWApSU1BVFhFSZ0Z0qj1QVZ2JZW5LczBhL2xGb0F5eW1uZU1CRBtGNG1XU1JmQng1cjY3Zm1lc4UcpWb0k0rMFA5R2RWUthJRDAXVvHdR2cdvY15WdQ13p6WGQwQmFqSk53QXNXUW5IR3pDak1SWFYKY1gzeXVWZH20LazHBnd1NNTDhNvj5da09TMeJdVGY3bE0yb000M1lyZVNMDApwQNdVSVJRbVfScVh3aFRbjTRFOULTQzhzNUZiQWdNQkFBR2pXVEJYTUE0R0ExVWREd0V1C1QWY4R0U1JGUURBUlUvGtUIwR0ExVWREZ1F0XkQJRRcUnzppWmmFMYnFslZrVNUdNmBIZH0UUFUFRYQYQmd0VkhSRUVEakFNZ2dwcMkRXSmxjbtVSzEdWek1TRQJYFR0UjPWRNRkZa0ZdNAppVURHSTMV0EzKwZbWtNkNzaxzhZm2R0VfvejZvGNbHEpUbjJhY0TRM0XVXW5GREU1c1lyI1wbNzYmVHQXh5C11uV0UdXk1OW9pGkKv0REXeYrF5Ym1lYmdS0TlmrXpSNz1PRGxUzINWqG5xdm0kYjRYdFZWVcsvg2TJRnREzNux1d0dWmkRhDnjYkYvJmJIM3j2RExpWfK1Wm1QNTNVMnbm0Z2HJZdo2VFBTKvORGXNdDhCER3a2hU0UJzdX1naGNIb3FOVKNbTMMNRkjJTQnFeXGsuSnpTbJx3Ck1ZcnBINXRI1FUNXW0E10b21Ta0drT2tsrcG1HSHNVMGfucm253NhEgKa1RSTNMCHRRM0txC0tLS0tRUSEIENFUR1Rk1DQVRFLS0tLS0K

server: https://EBFD13B05DA0171EA96228C7F6E79CFD.yl4.eu-west-2.eks.amazonaws.com
name: demo-cluster
eu-west-2.eksctl.io
contexts:
- context:
    cluster: demo-cluster
    eu-west-2.eksctl.io

```

And I copy that one which is already base64 encoded:

```
- cluster:
  certificate-authority-data: |LS0tLS1CRUdJTIhDVRVJUZjQ0FURS0tLS0tZm13SURCVEN0dWUyZ0F3SUJBRzJlYXZra1dQTGV0QV93RFR3cStvkw1odmNOQVFVEtJRQXdGVEV
  UTUJFRF0eXVUUKQXhNS2eZvYmlaWep1WlH5bSGNEQWGDzB5TpmBM1Ea3dPREKk7RWYUz3Mh6PaQeYtURDz09ESTVORFZHTJ3VeApFekF5QmdQvK3JTBVRDzXQxwVj6zBjAwE132ZdFdu
  1BMEduD3FHU0l1m0RRRUJBUVVBQTRJQRk3QxdZ0VLCKfVSUJBUURHtm1XtdnzB5VVk9tQU1L0DZ5Nk53dnkdEmrTVRRb0NkbgXqadEi4VGJFdXp1Nfg5UUV3WcUyQ0skTU15LUR0aGc1Ga
  FlScTSvZnuY2ZzGQ01sNVNncEvnL3pZlRNBzKfKtFXFNpPWFkVzljddjJRG1Scxkkzn1OWapsUULBVfH5SZ0ZQjqlQVZ2ZJW5LczBhL2XG0b5Ew1uZHFRWk4YlZAxOUdSdvg2aDQXhUdR
  RnEMG1UmlJm2qj1Cj3YznLmNhc4clpWb0KfA5R3RUWtJRDxAVWHDRCvdyY15MDL3pGWQ0qMfQsk3J5QXNNUW5R33pdkLSWFYKY1gzeXVWZH0b1FCNUHxTgTtsnQ4Vjn1Q0JENVV
  aZHBnd1NRTHJNUjd5a09TUEmDzG1VW92B0eY0Q0M1L1ZMDAdpQWdVSJ3RVfScvH3aGfRjTbFRUOL1TqzhZNU1QWdQmKfBR2pXVEJ3Tm008eXVWREd0VCL3dRRUF35UNWREFFQCk3nTlZ1Uk
  1CQNY4RQ3UQRUBUy0TUIwR0E4VGR3eZVjE1FXQJjRRE4XUWmFmFYn3L3RVNfUdEmPBjZHM0V4UyQVYQKQ0dVkhSRUeAvkFNZ2dwcwR3YX50b3vTS5eUk1BMEduD3FHU0l1m0RRRUdJr1VBQ
  TRJQkFRQWpNRWk2a0dZnAPPURH5TM0EQZcx7kTnkNZazhp3Zn2R0VjzeJbVGnH0eBujbJl4Rm0XVXWESGREU1Cllyr1lWbXN2YmVHQX5Cl1V0ZwXk1ADU040TVMdVf6Qz0u4DURC2
  0W9G0kV0REXxeF5Bvml1YmD5t0lmrXpNzS1PRGXU2WnqGWSdWokVjYrYdZWVCsv2TJRREXNz1d0dMkRHdn1ybKvjVmJIM3jTReXPWFk1Wm1QNTNVNw1VLVUWRnBTUnP85MFWbgFVTl
  0m0Z2HJURz0dZVbTKVORGXNDRIC3a2dXU0JZdXlNaGn1G13f0dKnbMTNRRj3TQnFexGusNpTbF3XkCZcnB2DhR8e1FUMW0zN10b21aTzBSXLmU19Q0UWUWREVRtFYq1k0Mn
  srcGIHSHNVMGfUcmZ3S3hXNkgKa1RzSTNMChRRM0tXi0tLS0tRU5EIENFUlRJRklDQVRFLS0tLS0k
  server: |E8FD13B05DA0171EA96228C7F6E79CFD.y14.eu-west-2.eks.amazonaws.com
  name: demo-cluster.eu-west-2.eksctl.net
```

We can see the **server** information as well so I could have grab that information from here as well 🙏

And paste the 'certificate-authority-data' in the 'config' file I ma creating for Jenkins:

```

apiVersion: v1
kind: Config
clusters:
- cluster:
    certificate-authority-data: LS0tLS1CRUdJTiBDRVJUSUZJQ0FUR50tLS0tCk1JSURCVENDQWUyZ0F3SUJBZ0lJYXZra1dQTGV0Q
FFTEJRQXdGEVUTUJFR0EXVUUKQXhNS2EzVmIaWEp1WlhSbGN6QWVGdzB5TmBmK1Ea3dPREkwTkRwYUz3MHP0akEyTURZd09ESTVORFZhTUJ
bXQxWW1WeWJtVjBaWE13Z2dFaU1BMEdDU3FHU0liM0RRRUJBUVVBQTRJQkR3QXdnZ0VLCKFvSUJBURhtMTlXTndzbW5VVK9TQUlLODZ5Nk5Jd
I4VGJFdXp1NFg5UUVJWCUUYQ0sKTU51VUR0aG1BaFlScTVsZnU2YzZGQW1sNVNNcEVnL3pFZ1RSNzFKTkFFNXpPWFkVz1jdj1JRG15ckxxZn1
VZ2JZW5LczBhL2xGb0F5eWluZHFhRWk4yLzAxOUFSdVg2aDQxUHQzRnBEMG1XULJmQng1CjY3ZmlnNHc4cUpWb0krMFA5RzRWUThJRDAxWVhDR
FqSkJ5QXNXUW5IR3pDak1SWFYKY1gzeXVWZHZ0b1FCNUhXTGttSnQ4VjNIQ0JENNVaZHBndLNNTDhNVjd5a09TMEdJVGy3bE0yb000MllyZVN
aFRjbTRFOU1TQzhzNUZiQWdNQkFBR2pXVEJYTUE0R0EXVWREd0VCL3dRRUF3SUNwREFQCKJnTLZiUk1CQWY4RUJUURBUUgVtUWwR0EXVWREZ
FSL2RVNUdmMnBIZHM0VUFUQVYKQmd0VkhSRUVEakFNZ2dwcwRXSmxjbTVsZEdWek1BMEdDU3FHU0liM0RRRUJDd1VBQTRJQkFRQWpNRWk2a0d
NkN2azhpZmU2R0VjeJZVbGNhbEpUbjBlaTRmOXVXWE5GREU1c1lyRlIwbXN2YmVHQXh5C1luV0UxdDAxU040TWVmVDF6QzQzaUdURCtZOW9GW
lmRXpSNz1PRGxUZWNqWG5xdWokYjRYdFZWVCsvT2JRRnEzNUx1d0dvMkRHdnJybKvJvMjIM3JtRExpWfK1Wm1QNTNVWX1VUzVWRnBTUnBPSmF
ZVFBTkVORGxNZDdhcER3a2hUOUJzdXlnaGNib3F0VkbNBTmNNRjJTQnFxeGxuSnpTbFJxck1ZcnBINXRIb1FUNWZoNE10b21Ta0drT2tBSXLUM
srcG1HSHNVMGFucmZvS3hXNEgKa1RzSTNMCHRRM0txCi0tLS0tRU5EIEFUF1RJRk1DQVRFLS0tLS0K
    server: https://EBFD13B05DA0171EA96228C7F6E79CFD.yl4.eu-west-2.eks.amazonaws.com
    name: kubernetes
contexts:
- context:
    cluster: kubernetes
    user: aws
    name: aws
current-context: aws
users:
- name: aws
  user:
    exec:
      apiVersion: client.authentication.k8s.io/v1beta1
      command: /usr/local/bin/aws-iam-authenticator
      args:
        - "token"
        - "-i"
-- INSERT (paste) --

```

And save it.

And check:

```

LICENSE README-zh_CN.md README.md config helmfile.tar.gz
root@jenkins-server:~# cat config
apiVersion: v1
kind: Config
clusters:
- cluster:
    certificate-authority-data: LS0tLS1CRUdJTiBDRVJUSUZJQ0FUR50tLS0tCk1JSURCVENDQWUyZ0F3SUJBZ0lJYXZra1dQTGV0Q
FFTEJRQXdGEVUTUJFR0EXVUUKQXhNS2EzVmIaWEp1WlhSbGN6QWVGdzB5TmBmK1Ea3dPREkwTkRwYUz3MHP0akEyTURZd09ESTVORFZhTUJ
bXQxWW1WeWJtVjBaWE13Z2dFaU1BMEdDU3FHU0liM0RRRUJBUVVBQTRJQkR3QXdnZ0VLCKFvSUJBURhtMTlXTndzbW5VVK9TQUlLODZ5Nk5Jd
I4VGJFdXp1NFg5UUVJWCUUYQ0sKTU51VUR0aG1BaFlScTVsZnU2YzZGQW1sNVNNcEVnL3pFZ1RSNzFKTkFFNXpPWFkVz1jdj1JRG15ckxxZn1
VZ2JZW5LczBhL2xGb0F5eWluZHFhRWk4yLzAxOUFSdVg2aDQxUHQzRnBEMG1XULJmQng1CjY3ZmlnNHc4cUpWb0krMFA5RzRWUThJRDAxWVhDR
FqSkJ5QXNXUW5IR3pDak1SWFYKY1gzeXVWZHZ0b1FCNUhXTGttSnQ4VjNIQ0JENNVaZHBndLNNTDhNVjd5a09TMEdJVGy3bE0yb000MllyZVN
aFRjbTRFOU1TQzhzNUZiQWdNQkFBR2pXVEJYTUE0R0EXVWREd0VCL3dRRUF3SUNwREFQCKJnTLZiUk1CQWY4RUJUURBUUgVtUWwR0EXVWREZ
FSL2RVNUdmMnBIZHM0VUFUQVYKQmd0VkhSRUVEakFNZ2dwcwRXSmxjbTVsZEdWek1BMEdDU3FHU0liM0RRRUJDd1VBQTRJQkFRQWpNRWk2a0d
NkN2azhpZmU2R0VjeJZVbGNhbEpUbjBlaTRmOXVXWE5GREU1c1lyRlIwbXN2YmVHQXh5C1luV0UxdDAxU040TWVmVDF6QzQzaUdURCtZOW9GW
lmRXpSNz1PRGxUZWNqWG5xdWokYjRYdFZWVCsvT2JRRnEzNUx1d0dvMkRHdnJybKvJvMjIM3JtRExpWfK1Wm1QNTNVWX1VUzVWRnBTUnBPSmF
ZVFBTkVORGxNZDdhcER3a2hUOUJzdXlnaGNib3F0VkbNBTmNNRjJTQnFxeGxuSnpTbFJxck1ZcnBINXRIb1FUNWZoNE10b21Ta0drT2tBSXLUM
srcG1HSHNVMGFucmZvS3hXNEgKa1RzSTNMCHRRM0txCi0tLS0tRU5EIEFUF1RJRk1DQVRFLS0tLS0K
    server: https://EBFD13B05DA0171EA96228C7F6E79CFD.yl4.eu-west-2.eks.amazonaws.com
    name: kubernetes
contexts:
- context:
    cluster: kubernetes
    user: aws
    name: aws
current-context: aws
users:

```

Now we need to make this file available inside the jenkins container. And it needs to be in the default kubeconfig location which is:
 .kube/config

So let's enter Jenkins container again as a Jenkins user:

-> docker exec -it 655647e3fb2e bash

```
root@jenkins-server:~# docker exec -it 655647e3fb2e bash
jenkins@655647e3fb2e:/$
```

Now let's move to Jenkins home directory

-> cd ~

And we can see the path to the home directory

-> pwd

```
jenkins@655647e3fb2e:~$ pwd
/var/jenkins_home
jenkins@655647e3fb2e:~$
```

We need to have .kube/ in the path /var/jenkins_home and we will move the config file we manually created in the droplet to the Jenkins container path /var/jenkins_home/.kube

So I need to have a directory called .kube

-> mkdir .kube

I already have it when I was playing with Jenkins after finishing Module 10 - chapter 24 !!

The way I played with Jenkins was creating the config file in the droplet and then I mounted it in the container, so I had to stop the container that was working and the time and remove it and create a new one with the additional volume configuration:

-> docker run -p 8080:8080 -p 50000:50000 -d -v /root/.kube:/var/jenkins_home/.kube -v jenkins_home:/var/jenkins_home jenkins/jenkins:its

```
root@jenkins-server:~# docker run -p 8080:8080 -p 50000:50000 -d -v /root/.kube:/var/jenkins_home/.kube -v jenkins_home:/var/jenkins_home jenkins/jenkins:its
655647e3fb2e2cc16169a39c762ceb1c6ad297b86619475a0196825c8840923d
root@jenkins-server:~# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
655647e3fb2e   jenkins/jenkins:its  "/usr/bin/tini -- /u..."  5 seconds ago  Up 4 seconds  0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp
root@jenkins-server:~#
```

I checked the container volumes:

-> docker inspect 655647e3fb2e | grep -A 10 Mounts

```

root@jenkins-server:~# docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS        PORTS
655647e3fb2e   jenkins/jenkins:lts "/usr/bin/tini -- /u... 2 weeks ago   Up 2 weeks   0.0.0.0:8
cp, 0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp   stupefied_mendel
root@jenkins-server:~# docker inspect 655647e3fb2e | grep -A 10 Mounts
  "Mounts": [
    {
      "Type": "bind",
      "Source": "/root/.kube",
      "Destination": "/var/jenkins_home/.kube",
      "Mode": "",
      "RW": true,
      "Propagation": "rprivate"
    },
    {
      "Type": "volume",

```

Your .kube folder is a bind mount — meaning:

```

/root/.kube/config on host
|
| IS THE SAME FILE AS
|
▼
/var/jenkins_home/.kube/config in container

```

What this means for copying a new config:

If the I copy a new config file into the container at `/var/jenkins_home/.kube/config` it will
overwrite `/root/.kube/config` on the host immediately.

So once I replace the config file from my previous demo with today's config file the config file in the droplet will change as well. This is ok as I no longer need the old config. But it is important to be careful **!!**

But first I am going to back up my old config

```
-> cp /root/.kube/config /root/.kube/config.backup
```

```
-> ls /root/.kube
```

```

root@jenkins-server:~# ls /root/.kube
cache  config  config.backup
root@jenkins-server:~#

```

So in my droplet I have:

- an old 'config' and its copy 'config.backup' `/root/.kube/` path
- and I have a new 'config' in `/root` path

```
root@jenkins-server:~# ls
LICENSE  README-zh_CN.md  README.md  config  helmfile.tar.gz
root@jenkins-server:~# pwd
/root
root@jenkins-server:~#
```

Now I will continue and I will copy the config file to Jenkins container:

-> docker cp config 655647e3fb2e:/var/jenkins_home/.kube/

Then get into jenkins container

-> docker exec -it 655647e3fb2e bash

Then cd into home directory

-> cd ~

Now check the config file

-> cat .kube/config


```

kind: Config
clusters:
- cluster:
    certificate-authority-data: LS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0tLS1U
    FFTEJRQXdGVEVUTUJFR0ExVUUKQXhNS2EzVmIaWEp1WlhSbGN6QWVGdzB5TmpBMk1
    bXQxWW1WeWJtVjBaWE13Z2dFaU1BMEdDU3FHU0liM0RRRUJBUVVBQTRJQkR3QXdnZ
    I4VGJFdxp1NFg5UVJWcUUYQ0sKTU5lVUR0aG1BaFlScTVsZnU2YzZGQW1sNVNNcEV
    V2ZJZW5LczBhL2xGb0F5eWluZHFWRk4yLzAxOUFSdVg2aDQxUHQzRnBEMG1XU1JmO
    FaSkJ5QXNXUW5IR3pDaklSWFYKY1gzeXVWZH0b1FCNUhXTGttSnQ4VjNIQ0JENV
    aFRjbTRFOU1TQzhzNUZiQWdNQkFBR2pXVEJYTUE0R0ExVWREd0VCL3dRRUF3SUNwF
    FsL2RVNUdmMnBIZHM0VUFUQVYKQmdOVkhSRUVEakFNZ2dwcmRXSmxjbTVsZEdWeK1
    NkN2azhpZmU2R0VjejZVbGNhbEpUbjbLaTRmOXVXWE5GREU1c1lyRlIwbXN2YmVH
    lmxRXPmSNz1PRGxUZWNqWG5xdWokYjRYdFZWVCsvT2JRRnEzNUx1d0dvMkRHdnJybk
    ZVFBTkVORGxNZDdhcER3a2hUOUJzdXlnaGNib3F0VknBTmNNRjJlJTQnFxeGxuSnP
    ThsrcG1HSHNVMGFucmZvS3hXNEgKa1RzSTNMchRRM0txCi0tLS0tRU5EIEENFU1RJRk
    server: https://EBFD13B05DA0171EA96228C7F6E79CFD.yl4.eu-west-
    name: kubernetes
contexts:
- context:
    cluster: kubernetes
    user: aws
    name: aws
current-context: aws
users:
- name: aws
  user:
    exec:
      apiVersion: client.authentication.k8s.io/v1beta1
      command: /usr/local/bin/aws-iam-authenticator
      args:
        - "token"
        - "-i"
        - "demo-cluster"

```

4. Credentials for AWS user

Create AWS credentials for a dedicated **Jenkins IAM user**, so when Jenkins kubectl runs aws-iam-authenticator it can use those credentials to authenticate with AWS and get a token to talk to EKS.

Dedicated IAM **user or role** for Jenkins:

Option 1 — dedicated IAM user for Jenkins (simpler):

create IAM user: jenkins-ci

|



attach only the permissions Jenkins needs:

- push/pull ECR images
- deploy to EKS
- read S3 (if needed)

|



generate access key + secret key for jenkins-ci

|



store in Jenkins Credentials Store (not hardcoded)

|



pipeline uses them at runtime

Option 2 — IAM role for Jenkins (better, if Jenkins runs on EC2/EKS):

attach IAM role directly to the EC2 instance or pod
running Jenkins

|



Jenkins automatically gets temporary credentials
no access keys needed at all
credentials rotate automatically

For your DigitalOcean droplet setup:

Since Jenkins runs on a DigitalOcean droplet (not AWS EC2), it can't use IAM roles directly. So Option 1 is your path:

AWS console

|



IAM → create user: jenkins-ci

|



attach permissions policy

|



generate access key + secret key

|



Jenkins UI → Manage Jenkins → Credentials

|



add AWS credentials:
- AWS_ACCESS_KEY_ID

- AWS_SECRET_ACCESS_KEY



pipeline references credentials by ID:

```
withCredentials([aws(credentialsId: 'jenkins-aws')]) {  
    sh 'aws eks get-token --cluster-name demo-cluster'  
}
```

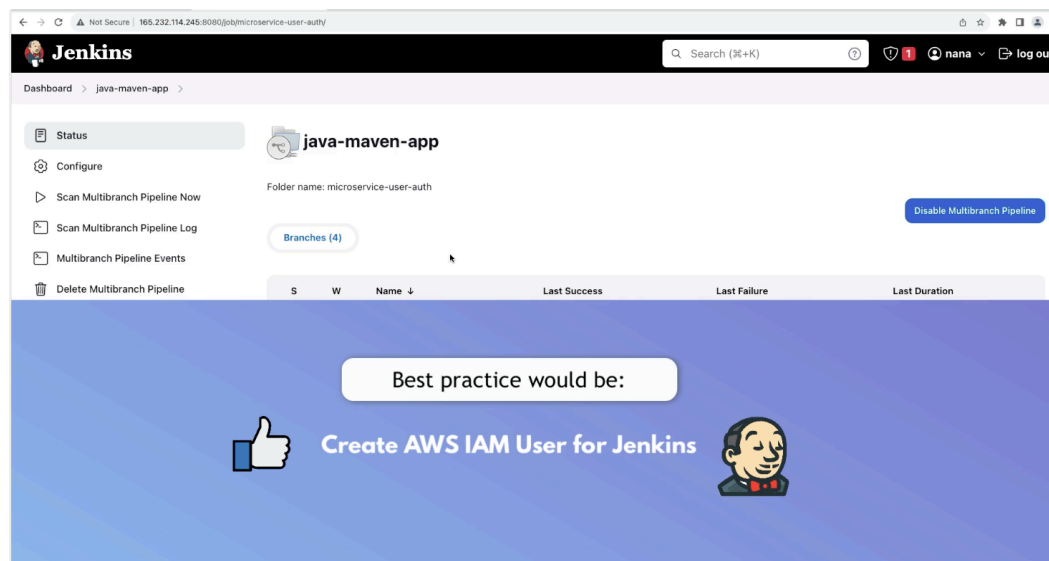
Least privilege for jenkins-ci IAM user:

Only give Jenkins what it actually needs:

Task	AWS Permission needed
Deploy to EKS	eks:DescribeCluster
Push to ECR	ecr:GetAuthorizationToken, ecr:PushImage
Pull from ECR	ecr:GetAuthorizationToken, ecr:PullImage
Read S3	s3:GetObject

Not admin, not full access — just what the pipeline needs to do its job.

So locally I have been using the user 'admin' and as our Jenkins is not in AWS but in digital Ocean we will implement the first approach which is to create a new AWS user specifically for Jenkins in AWS:



In AWS Console:

AWS Console → IAM → Users → Create User

Step 1 — name the user:

username: jenkins

Step 1 Specify user details

Step 2 Set permissions

Step 3 Review and create

Specify user details

User details

User name

jenkins

The user name can have up to 64 characters. Valid characters: A-Z, a-z, 0-9, and +, -, @, _ (hyphen)

☐ Provide user access to the AWS Management Console - optional

In addition to console access, users with `SigninLocalDevelopmentAccess` permissions can use the same console credentials for programmatic access without the need for access keys.

[Learn more](#)

Cancel Next

Step 2 — set permissions:

Attach policies directly



for now to get started you can attach:

AmazonEKSClusterPolicy

AmazonEC2ContainerRegistryFullAccess ← if using ECR

Or create a custom policy with least privilege later.

Step 1 Specify user details

Step 2 Set permissions

Step 3 Review and create

Set permissions

Add user to an existing group or create a new one. Using groups is a best-practice way to manage user's permissions by job functions. [Learn more](#)

Permissions options

☐ Add user to group

Add user to an existing group, or create a new group. We recommend using groups to manage user permissions by job function.

☐ Copy permissions

Copy all group memberships, attached managed policies, and inline policies from an existing user.

☒ Attach policies directly

Attach a managed policy directly to a user. As a best practice, we recommend attaching policies to a group instead. Then, add the user to the appropriate group.

Permissions policies (2/1502)

Choose one or more policies to attach to your new user.

Filter by Type: All types 1 match

Policy name	Type	Attached entities
☒ AmazonEC2ContainerRegistryFullAccess	AWS managed	0

AmazonEC2ContainerRegistryFullAccess

Provides administrative access to Amazon ECR resources

```
6+  "Action": [
7    "ecr:*",
8    "cloudtrail:LookupEvents"
9  ],
10  "Resource": "*"
11 },
12 {
13   "Effect": "Allow",
14   "Action": [
15     "iam:CreateServiceLinkedRole"
16   ],
17   "Resource": "*",
18   "Condition": {
19     "StringEquals": {
20       "iam:AWSServiceName": [
21         "replication-ecr.amazonaws.com"
22       ]
23     }
24   }
25 }
```

Review and create
Review your choices. After you create the user, you can view and download the autogenerated password, if enabled.

User details

User name jenkins	Console password type None	Require password reset No
----------------------	-------------------------------	------------------------------

Permissions summary

Name	Type	Used as
AmazonEC2ContainerRegistryFullAccess	AWS managed	Permissions policy
AmazonEKSClusterPolicy	AWS managed	Permissions policy

Tags - optional
Tags are key-value pairs you can add to AWS resources to help identify, organize, or search for resources. Choose any tags you want to associate with this user.
No tags associated with the resource.
[Add new tag](#)
You can add up to 50 more tags.

[Cancel](#) [Previous](#) [Create user](#)

Limited permissions:

User created successfully
You can view and download the user's password and email instructions for signing in to the AWS Management Console.
[View user](#)

IAM users (3)

An IAM user is an identity with long-term credentials that is used to interact with AWS in an account.

☐	User name	Path	Group	Last activity	MFA	Password age	Console last sign-in	Access key ID	Active key age	Access key last use	ARN
☐	admin	/	0	4 hours ago	-	77 days	4 hours ago	Active - AKIA2JGYSVB...	77 days	4 hours ago	arn:aws:iam::arrrrrrrrr:iam/user/admin
☐	jenkins	/	0	-	-	-	-	-	-	-	arn:aws:iam::arrrrrrrrr:iam/user/jenkins
☐	MyUserCLI	/	1	41 days ago	-	41 days	41 days ago	Active - AKIA2JGYSVB...	41 days	41 days ago	arn:aws:iam::arrrrrrrrr:iam/user/MyUserCLI

Step 3 — create access keys:

IAM → Users → jenkins



Security credentials tab



Access keys → Create access key



use case: select "Application running outside AWS"



AWS generates:

AWS_ACCESS_KEY_ID → copy this

AWS_SECRET_ACCESS_KEY → copy this (only shown once)

[IAM](#) > [IAM users](#) > [jenkins](#)

entity and Access management (IAM)

[shboard](#)

[zess Management](#)

[ies](#)

[licies](#)

[t users](#)

[t user groups](#)

[ntity providers](#)

[count settings](#)

[ot access management](#)

[mportary delegation requests](#)

[zess reports](#)

[ress Analyzer](#)

[Resource analysis](#)

[Unused access](#)

[Analyzer settings](#)

[idental report](#)

[ganization activity](#)

[rice control policies](#)

[source control policies](#)

[entof console](#)

[t IAM Center](#)

[S Organizations](#)

jenkins

Summary

ABN

arn:aws:iam::706976333940:user/jenkins

Console access

Disabled

Access key 1

[Create access key](#)

Created

June 09, 2026, 14:23 (UTC+01:00)

Last console sign-in

-

Permissions

Groups

Tags

Security credentials

Last Accessed

Console sign-in

Console sign-in link

https://706976333940.signin.aws.amazon.com/console

Console password

Not enabled

Enable console access

Multi-factor authentication (MFA) (0)

Remove

Reassign

Assign MFA device

Use MFA to increase the security of your AWS environment. Signing in with MFA requires an authentication code from an MFA device. Each user can have a maximum of 8 MFA devices assigned. [Learn more](#)

Type

Identifier

Certifications

Created on

No MFA devices. Assign an MFA device to improve the security of your AWS environment

Assign MFA device

Access keys (0)

Generate access key

Use access keys to send programmatic calls to AWS from the AWS CLI, AWS Tools for PowerShell, AWS SDKs, or direct AWS API calls. You can have a maximum of two access keys (active or inactive) at a time. [Learn more](#)

No access keys. As a best practice, avoid using long-term credentials like access keys. Instead, use tools which provide short term credentials. [Learn more](#)

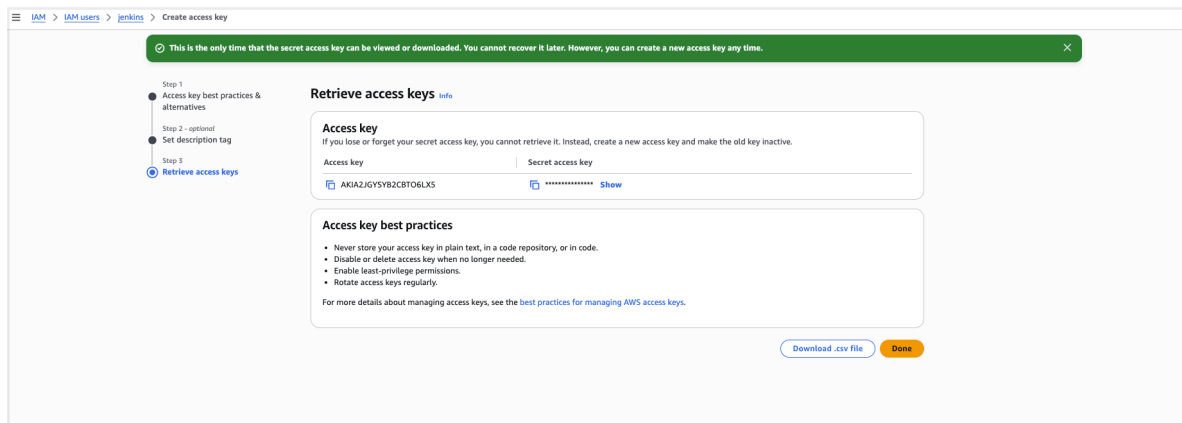
Create access key

API keys (0)

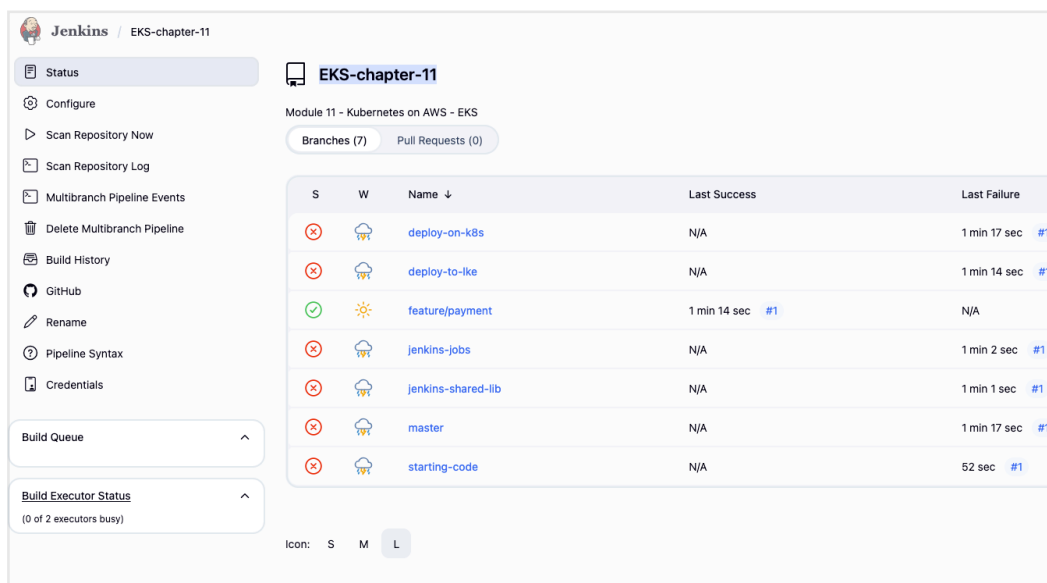
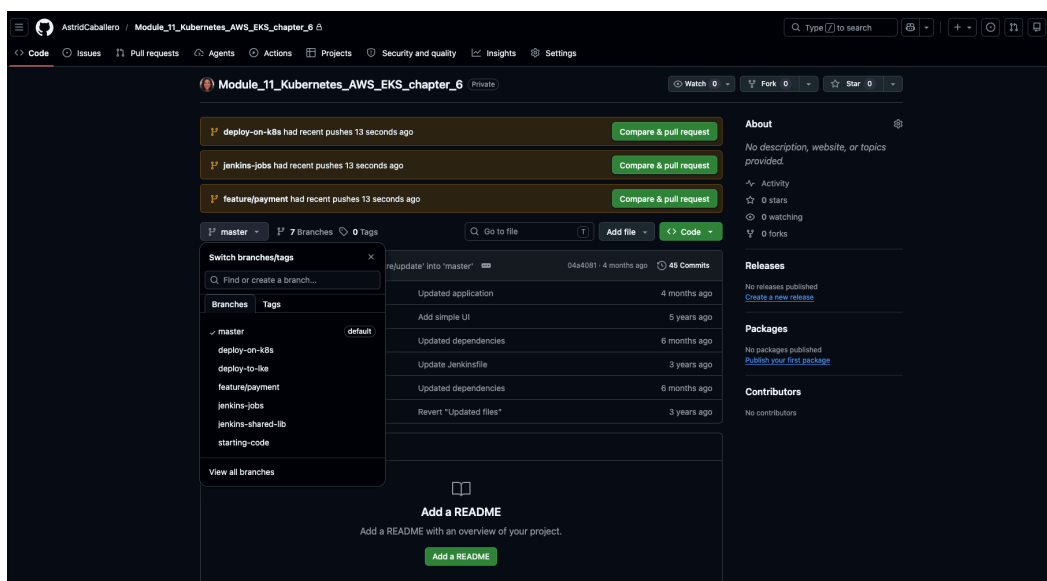
Actions

Generate API Key

API keys grant access to specific AWS services. You can have a maximum of two API keys per service (active, inactive, or expired) at a time. [Learn more](#)



I create a new Multibranch Pipeline in Jenkins and a new GitHub repo https://github.com/AstridCaballero/Module_11_Kubernetes_AWS_EKS_chapter_6



The builds above are failing because of the credentials, for example deploy-on-k8s:

```
Jenkins / EKS-chapter-11 / deploy-on-k8s / #1 / Console Output

[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (build app)
[Pipeline] script
[Pipeline] {
[Pipeline] echo
[Pipeline] building the application...
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (build image)
[Pipeline] script
[Pipeline] {
[Pipeline] echo
[Pipeline] building the docker image...
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (deploy)
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
ERROR: jenkins_aws_access_key_id

GitHub has been notified of this commit's build result

Finished: FAILURE
```

Jenkinsfile in that branch looks for the access key and the secret (line 22 and 23) and we don't have those credentials in Jenkins yet:

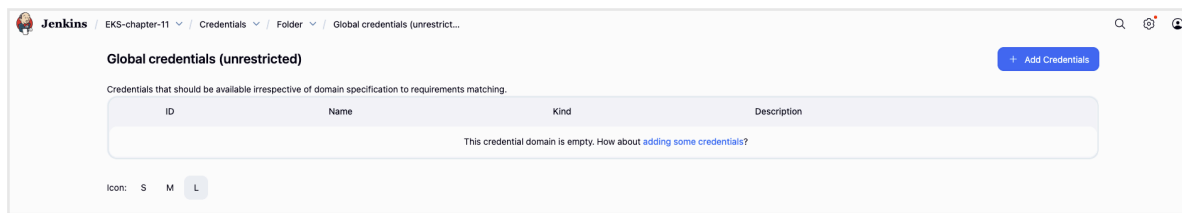
```
Module_11_Chapter_6 / Jenkinsfile
Project
  Module_11_Chapter_6 [java-maven-app]
    .idea
    src
    .gitignore
    Dockerfile
    Jenkinsfile
    pom.xml
  External Libraries
  Scratches and Consoles

Jenkinsfile
16
17
18
19
20
21
22 AWS_ACCESS_KEY_ID = credentials('jenkins_aws_access_key_id')
23 AWS_SECRET_ACCESS_KEY = credentials('jenkins-aws_secret_access_key')
24
25
26
27
28
29
script {
    echo "building the docker image..."
}
}
}
stage('deploy') {
    environment {
22 AWS_ACCESS_KEY_ID = credentials('jenkins_aws_access_key_id')
23 AWS_SECRET_ACCESS_KEY = credentials('jenkins-aws_secret_access_key')
    }
    steps {
        script {
27 echo 'deploying docker image...'
28 sh 'kubectl create deployment nginx-deployment --image=nginx'
29
        }
    }
}
```

note: Nana used the credentials for her user 'admin' as a pragmatical choice for the demo but she mentioned is bad practice. So I created a user specifically for Jenkins (which has fewer permission than 'admin') and Nana then get's the access key and the secrete from her local computer (I got it directly from AWS console)

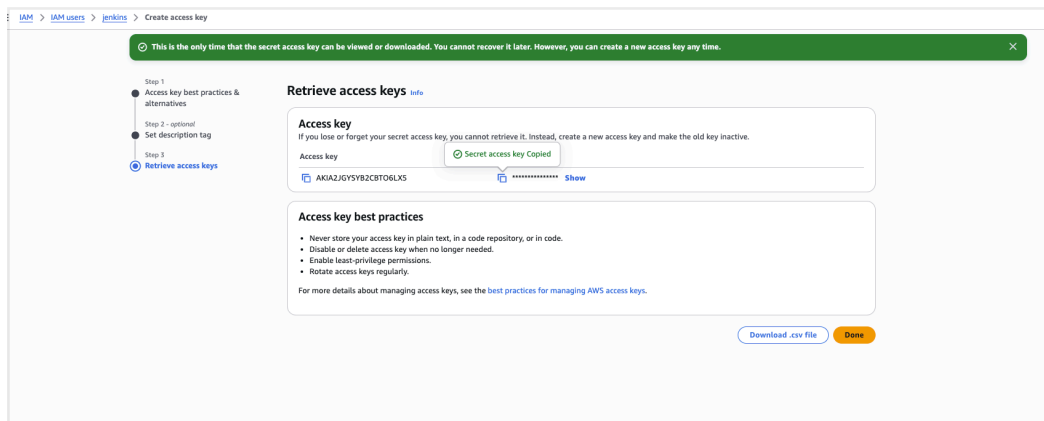
-> cat .aws/credentials

I now create credentials for this pipeline 'EKS-chapter-11':

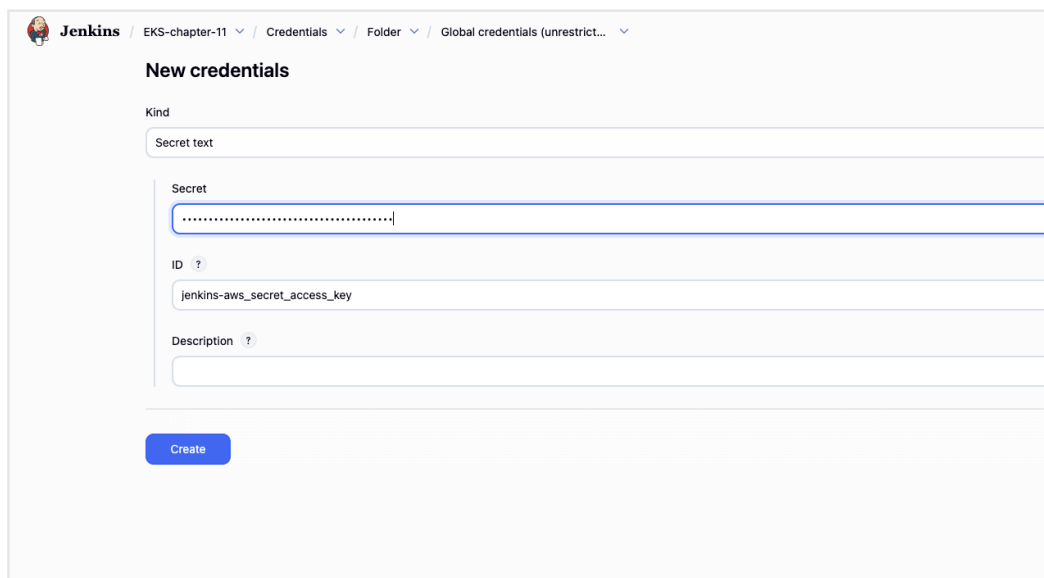


Of type 'secret text' for both the access key and the secret:

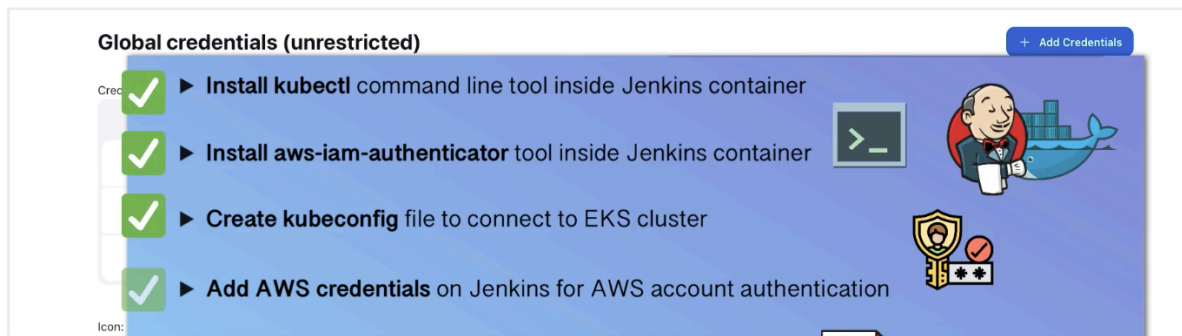
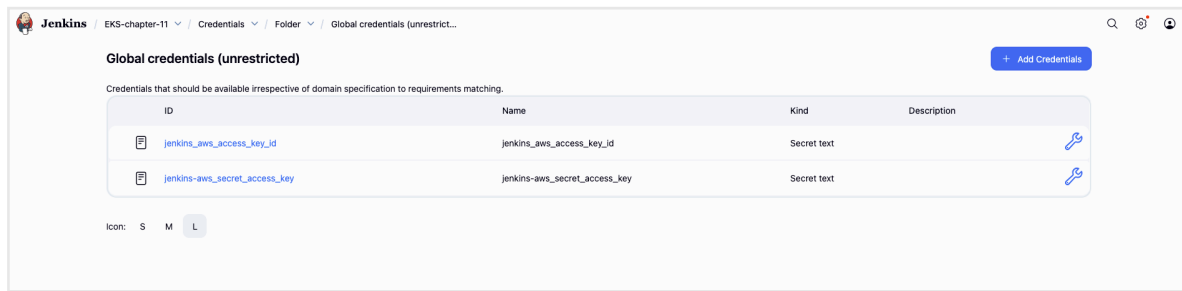
Copy from here:



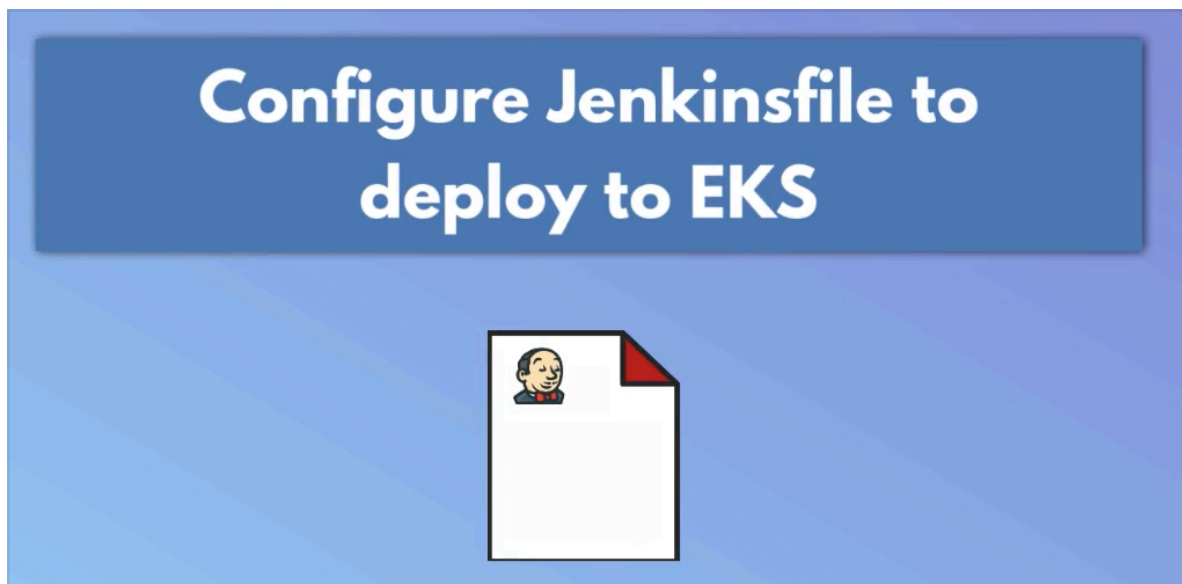
To here:



And now I have both credentials



5. Create/Adjust Jenkinsfile to configure EKS cluster deployment



Now Jenkins can execute kubectl:

```
-> sh 'kubectl create deployment nginx-deployment --image=nginx'
```

```

25     steps {
26     script {
27         echo 'deploying docker image...'
28         sh 'kubectl create deployment nginx-deployment --image=nginx'
29     }
30 }
31 }

```

But before that we need to set the credentials as environment variables (line 21-24);

```

20 stage('deploy') {
21     environment {
22         AWS_ACCESS_KEY_ID = credentials('jenkins_aws_access_key_id')
23         AWS_SECRET_ACCESS_KEY = credentials('jenkins-aws_secret_access_key')
24     }
25     steps {
26     script {
27         echo 'deploying docker image...'
28         sh 'kubectl create deployment nginx-deployment --image=nginx'
29     }
30 }
31 }
32 }

```

As the credentials are now env vars, kubectl can use it when executing aws-iam-authenticator.

Locally, in my laptop I have my credentials in a file called config:
-> aws configure list

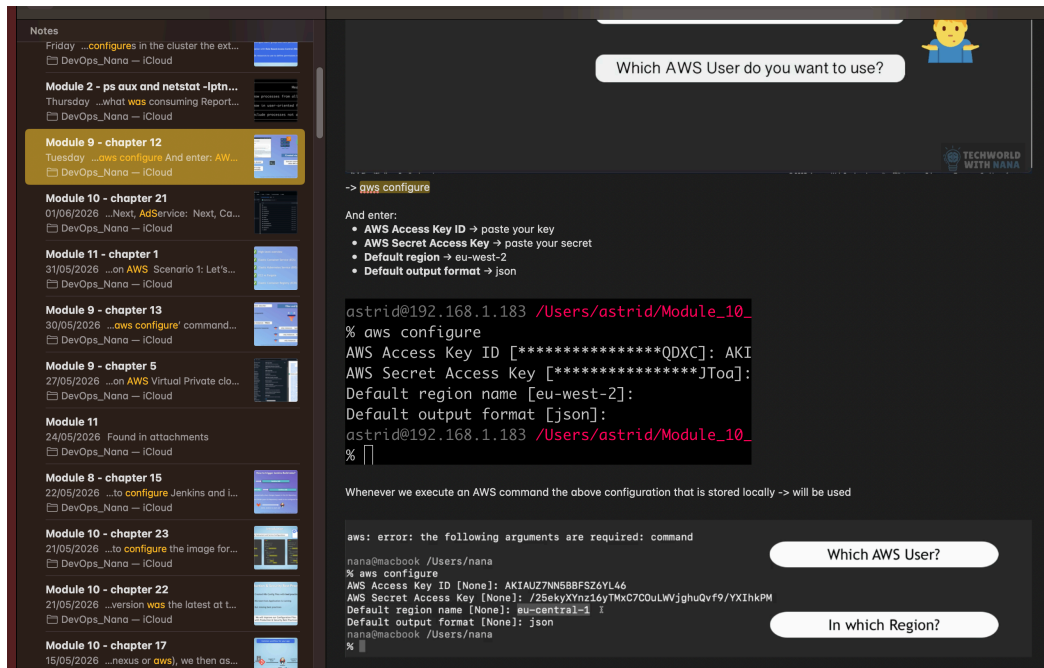
```

% aws configure list
NAME      : VALUE                                : TYPE      : LOCATION
profile   : <not set>                             : None      : None
access_key : *****US47                          : shared-credentials-file :
secret_key : *****j+m6                          : shared-credentials-file :
region    : eu-west-2                            : config-file   : ~/.aws/config
astrid@192.168.1.112 /Users/astrid
%

```

The above was configured in Module 9 - chapter 12 with
-> aws configure

And it stored the configuration in a config file.



We could have created env vars in each terminal window, in each context, instead.

For Jenkins, we don't have the ~/.aws/credentials file so we store the AWS credentials in the Jenkins Credentials Store and inject them as environment variables into the pipeline when the Jenkinsfile runs.

credentials stored encrypted in Jenkins



injected temporarily as env vars only during pipeline execution



not visible outside of that pipeline run

The credentials never live permanently in the container — they are injected only when needed and only for that specific pipeline run.

Q:

I see, and the pipeline is running in the container, it is process inside the container, and when is done everything used by the pipeline is gone.

A:

Yes exactly.

Jenkins container running



pipeline triggered (a process starts inside the container)

↓
▼
Jenkins injects AWS credentials as env vars
into that process only

↓
▼
pipeline runs:
- fetches code
- builds image
- pushes to ECR
- deploys to EKS

↓
▼
pipeline process finishes

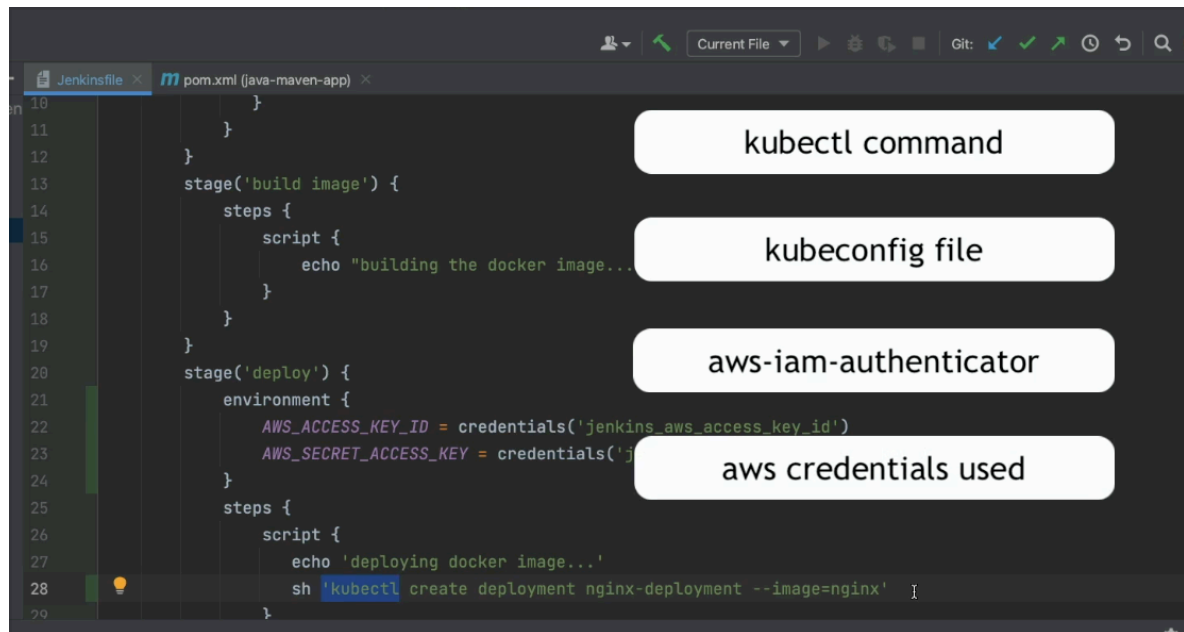
↓
▼
env vars gone
credentials no longer in memory

The credentials only exist for the lifetime of that pipeline process — they are not stored anywhere in the container permanently.

This is the same principle we discussed earlier:

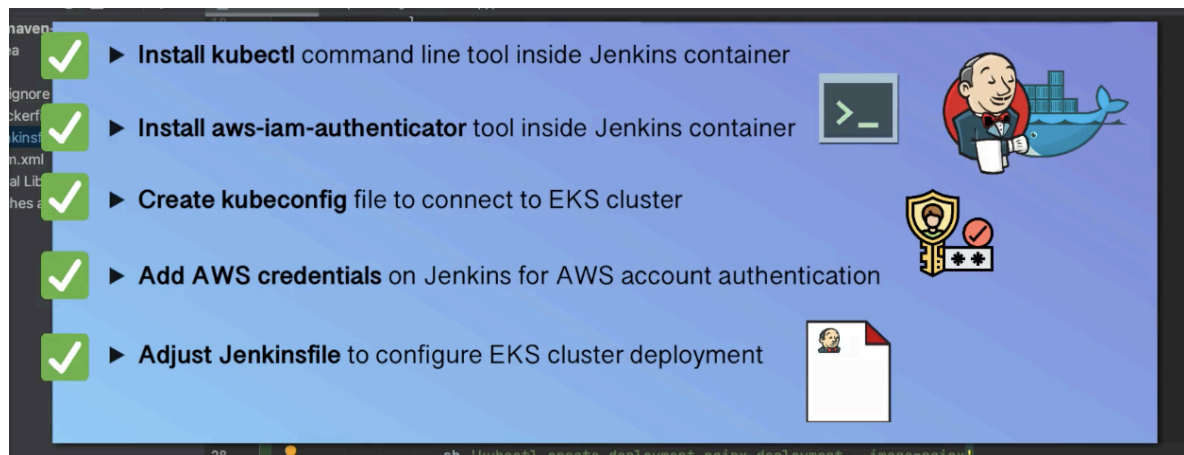
- process starts → resources allocated (env vars, memory)
- process ends → everything released and gone

The Jenkins container itself keeps running waiting for the next pipeline trigger, but the credentials from the previous run are completely gone.



The screenshot shows a Jenkinsfile in a code editor. The file contains two stages: 'build image' and 'deploy'. The 'deploy' stage has an environment section with AWS credentials and a script step that runs 'kubectl create deployment nginx-deployment --image=nginx'. Four white callout boxes with black text point to specific parts of the file: 'kubectl command' points to the 'kubectl' command in the script; 'kubeconfig file' points to the 'AWS_ACCESS_KEY_ID' and 'AWS_SECRET_ACCESS_KEY' credentials; 'aws-iam-authenticator' points to the 'AWS_SECRET_ACCESS_KEY' credential; and 'aws credentials used' points to the 'AWS_ACCESS_KEY_ID' credential.

```
10 }
11 }
12 }
13 stage('build image') {
14   steps {
15     script {
16       echo "building the docker image..."
17     }
18   }
19 }
20 stage('deploy') {
21   environment {
22     AWS_ACCESS_KEY_ID = credentials('jenkins_aws_access_key_id')
23     AWS_SECRET_ACCESS_KEY = credentials('jenkins_aws_secret_access_key')
24   }
25   steps {
26     script {
27       echo 'deploying docker image...'
28       sh 'kubectl create deployment nginx-deployment --image=nginx'
29     }
30   }
31 }
```

- 
- A checklist of five steps for deploying an EKS cluster, each preceded by a green checkmark icon. The steps are: 1. Install kubectl command line tool inside Jenkins container (with a terminal icon); 2. Install aws-iam-authenticator tool inside Jenkins container (with a terminal icon); 3. Create kubeconfig file to connect to EKS cluster (with a key icon); 4. Add AWS credentials on Jenkins for AWS account authentication (with a key icon); 5. Adjust Jenkinsfile to configure EKS cluster deployment (with a document icon). The background is a blue gradient with a cartoon character of a man in a suit holding a briefcase.
- ✓ ▶ Install kubectl command line tool inside Jenkins container
 - ✓ ▶ Install aws-iam-authenticator tool inside Jenkins container
 - ✓ ▶ Create kubeconfig file to connect to EKS cluster
 - ✓ ▶ Add AWS credentials on Jenkins for AWS account authentication
 - ✓ ▶ Adjust Jenkinsfile to configure EKS cluster deployment

Q:

why for jenkins, the sh cammand is -> 'kubectl create deployment nginx-deployment --image=nginx'
and not kubectl -> apply -f nginx-deployment.yaml

A:

Good observation — both work, it's a choice. But here's why sh with a direct command is common in Jenkins pipelines:

kubectl create vs kubectl apply -f:

kubectl create deployment nginx --image=nginx
→ **imperative**: tell k8s exactly what to do

kubectl apply -f nginx-deployment.yaml
→ **declarative**: give k8s a desired state file

Why pipelines often use the imperative approach:

Simplicity – no need to manage a yaml file in the pipeline:

```
// imperative - one line, no file needed
sh 'kubectl create deployment nginx --image=nginx'
```

```
// declarative - needs yaml file in repo
sh 'kubectl apply -f nginx-deployment.yaml'
```

Dynamic values – pipelines often need to inject variables like image tags:

```
// easy to inject variables imperatively
sh "kubectl set image deployment/nginx nginx=${IMAGE_NAME}:${BUILD_NUMBER}"
```

```
// with yaml you'd need to template the file first
sh "sed -i 's/IMAGE_TAG/${BUILD_NUMBER}/' nginx-deployment.yaml"
sh 'kubectl apply -f nginx-deployment.yaml'
```

However in real production pipelines apply -f is preferred:

imperative (create):

- fails if resource already exists
- harder to track what's deployed
- not idempotent

declarative (apply -f):

- works whether resource exists or not
- yaml file is version controlled
- idempotent (safe to run multiple times)
- infrastructure as code

So the honest answer:

The teacher likely used kubectl create in the Jenkinsfile to keep it simple for learning. In real production pipelines you would use kubectl apply -f with yaml files stored in the repository — that's the proper GitOps approach.

Let's execute the Jenkins Pipeline



Execute Jenkins Pipeline

Let's build only the branch 'deploy-on-k8s' so we need to edit the configuration

os the pipeline:

The screenshot shows the Jenkins Pipeline Configuration page. It has a sidebar on the left and a main configuration area on the right. The main area is divided into sections: 'Filter by name (with regular expression)', 'Property strategy', and 'Build strategies'. The 'Filter by name' section has a 'Regular expression' input field containing 'deploy-on-k8s'. Below it is an 'Add' button with a dropdown arrow. The 'Property strategy' section has a dropdown menu showing 'All branches get the same properties' and a '+ Add property' button. The 'Build strategies' section is currently empty. At the bottom of the configuration area are two buttons: 'Save' (in blue) and 'Apply' (in grey).

I restarted jenkins container when installing a plugin. Because of that I lost the previous jenkins container and I had to run anew one:

```
-> docker run -p 8080:8080 -p 50000:50000 -d  
-v /root/.kube:/var/jenkins_home/.kube  
-v jenkins_home:/var/jenkins_home  
jenkins/jenkins:its
```

The re run the job and got:

```
-> + kubectl create deployment nginx-deployment --image=nginx  
/var/jenkins_home/workspace/EKS-chapter-11_deploy-on-k8s@tmp/  
durable-7c425c25/script.sh.copy: 1: kubectl: not found
```

The screenshot shows the Jenkins Pipeline console output. It displays the following lines: '[Pipeline] echo', 'deploying docker image...', '[Pipeline] sh', '+ kubectl create deployment nginx-deployment --image=nginx', '/var/jenkins_home/workspace/EKS-chapter-11_deploy-on-k8s@tmp/durable-7c425c25/script.sh.copy: 1: kubectl: not found', '[Pipeline] }', '[Pipeline] // script', '[Pipeline] }', and '[Pipeline] // withCredentials'. The error message is highlighted in blue.

And now I lost kubectl, it isn't found (I didn't lose kube config file because I had it in root and I mounted it in the container, so the config file is stored in the droplet so as long as the droplet doesn't restart the container will be fine) and I also lost aws-iam-authenticator

```
-> ls /var/jenkins_home/.kube/
```



```
jenkins@ec090f856ef8:/$ ls /var/jenkins_home/.kube/  
cache config config.backup
```

So I installed kubectl in the droplet:

```
-> curl -LO https://storage.googleapis.com/kubernetes-release/release/$(curl  
-s https://storage.googleapis.com/kubernetes-release/release/stable.txt)/bin/  
linux/amd64/kubectl; chmod +x ./kubectl; mv ./kubectl /usr/local/bin/kubectl  
-> ls -a /usr/local/bin
```

```
root@jenkins-server:~# ls -a /usr/local/bin  
.  
..  
root@jenkins-server:~# curl -LO https://storage.googleapis.com/kubernetes-release/release/$(curl -s https://storage.googleapis.com/  
/kubernetes-release/release/stable.txt)/bin/linux/amd64/kubectl; chmod +x ./kubectl; mv ./kubectl /usr/local/bin/kubectl  
% Total % Received % Xferd Average Speed Time Time Time Current  
Dload Upload Total Spent Left Speed  
100 53.7M 100 53.7M 0 0 29.1M 0 0:00:01 0:00:01 --:--:-- 29.1M  
root@jenkins-server:~# ls -a /usr/local/bin  
.  
..  
kubectl
```

And Installed aws-iam-authenticator in the droplet:

```
-> curl -Lo aws-iam-authenticator https://github.com/kubernetes-sigs/aws-iam-authenticator/releases/download/v0.6.11/aws-iam-authenticator\_0.6.11\_linux\_amd64  
-> chmod +x ./aws-iam-authenticator  
-> mv ./aws-iam-authenticator /usr/local/bin  
-> ls /usr/local/bin/
```

```
root@jenkins-server:~# curl -Lo aws-iam-authenticator https://github.com/kubernetes-sigs/aws-iam-authenticator/releases/download/v  
0.6.11/aws-iam-authenticator_0.6.11_linux_amd64  
% Total % Received % Xferd Average Speed Time Time Time Current  
Dload Upload Total Spent Left Speed  
0 0 0 0 0 0 0 0 0:00:00 0:00:00 0:00:00 0  
100 51.0M 100 51.0M 0 0 38.5M 0 0:00:01 0:00:01 --:--:-- 45.4M  
root@jenkins-server:~# chmod +x ./aws-iam-authenticator  
root@jenkins-server:~# ls /usr/local/bin  
kubectl  
root@jenkins-server:~# mv ./aws-iam-authenticator /usr/local/bin  
root@jenkins-server:~# ls /usr/local/bin  
aws-iam-authenticator kubectl
```

And mount them onto jenkins container for the future:

```
docker run -p 8080:8080 \  
-p 50000:50000 -d \  
-v /root/.kube:/var/jenkins_home/.kube \  
-v /usr/local/bin/kubectl:/usr/local/bin/kubectl \  
-v /usr/local/bin/aws-iam-authenticator:/usr/local/bin/aws-iam-authenticator \  
-v jenkins_home:/var/jenkins_home \  
jenkins/jenkins:its
```

Pipeline still failing, the error now is:

```
-> + kubectl create deployment nginx-deployment --image=nginx  
error: failed to create deployment: Unauthorized
```

```
[Pipeline] sh
+ kubectl create deployment nginx-deployment --image=nginx
error: failed to create deployment: Unauthorized
[Pipeline] }
[Pipeline] // script
```

Problem:

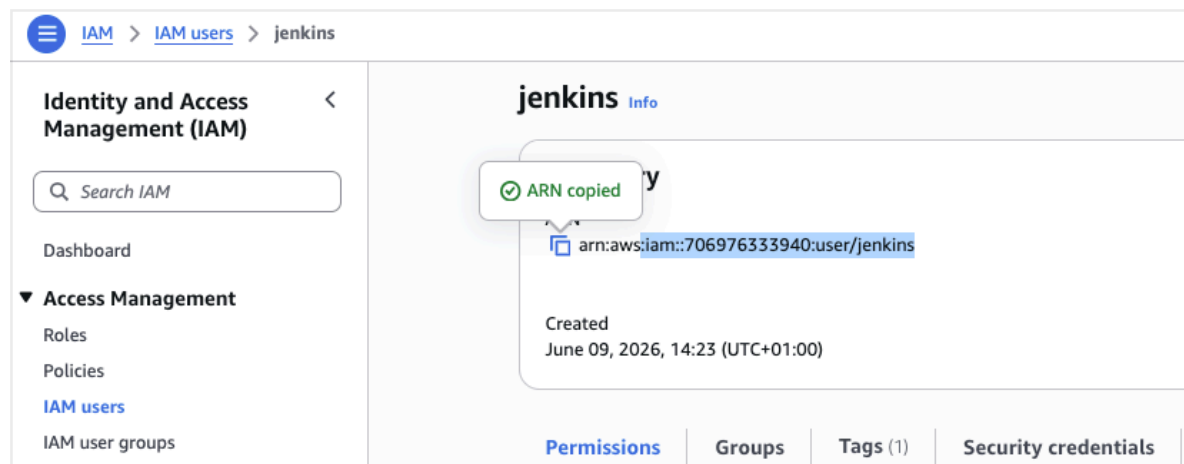
Check my aws account from my laptop:

-> kubectl get configmap aws-auth -n kube-system -o yaml

The aws-auth ConfigMap only has the node group role, no Jenkins IAM user.
That's why EKS rejects the Jenkins credentials.

```
% kubectl get configmap aws-auth -n kube-system -o yaml
apiVersion: v1
data:
  mapRoles: |
    - rolearn: arn:aws:iam::706976333940:role/eksctl-demo-cluster-nodegroup-demo-NodeInstanceRole-vFVE9tMraBd9
      groups:
        - system:bootstrappers
        - system:nodes
      username: system:node:{{EC2PrivateDNSName}}
kind: ConfigMap
metadata:
  creationTimestamp: "2026-06-09T08:35:43Z"
  name: aws-auth
  namespace: kube-system
  resourceVersion: "1151"
  uid: 28c70cb9-0ec1-4349-a0b7-5327573a7069
```

So I copy my Jenkins ARN:



Then edit the **aws-auth** ConfigMap:

-> kubectl edit configmap aws-auth -n kube-system

This opens vim. Add the mapUsers section so it looks like this:

```
mapUsers: |
  - userarn: arn:aws:iam::706976333940:user/jenkins
    username: jenkins
    groups:
      - system:masters
```

of



The screenshot displays the Jenkins web interface for a job named "deploy-on-k8s". The left sidebar contains navigation links: Status, Changes, Build Now, View Configuration, Full Stage View, Stages, GitHub, Pipeline Syntax, and Credentials. The main panel shows the "Stage View" for the selected job.

Job Information:

- Full project name: EKS-chapter-11/deploy-on-k8s

Average stage times: (full run time: ~5s)

	Declarative: Checkout SCM	build app	build image	deploy
#5 Jun 09 17:50 No Changes	1s	332ms	224ms	841ms
#4 Jun 09 17:40 1 commit	1s	202ms	216ms	945ms failed
#3 Jun 09 17:26 No Changes	1s	385ms	183ms	1s failed
#2 Jun 09 16:50 No Changes	2s	313ms	214ms	1s failed
#1 Jun 09 15:21 No Changes	2s	486ms	287ms	272ms failed

Built Builds Section:

Today

- #5 4:50 PM ✓
- #4 4:40 PM ✗
- #3 4:26 PM ✗
- #2 3:50 PM ✗
- #1 2:21 PM ✗

Permalinks section:

- Last build (#4), 9 min 22 sec ago
- Last failed build (#4), 9 min 22 sec ago
- Last unsuccessful build (#4), 9 min 22 sec ago
- Last completed build (#4), 9 min 22 sec ago

```

[Pipeline] sh
+ kubectl create deployment nginx-deployment --image=nginx
deployment.apps/nginx-deployment created
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline

```

GitHub has been notified of this commit's build result

Finished: SUCCESS

-> kubectl get pod

```

astrid@192.168.1.112 /Users/astrid
% kubectl get pod
NAME                                READY   STATUS    RESTARTS   AGE
nginx-deployment-6ff797d4c9-zvrhq  0/1     Pending   0           9m30s
astrid@192.168.1.112 /Users/astrid
%

```

I decided to delete the cluster at the end of the chapter:

-> eksctl delete cluster --name demo-cluster

One thing to be **aware of** — after deletion your kubeconfig will still have the entry for demo-cluster

pointing at a non-existent endpoint. Clean it up afterwards:

```
kubectl config delete-context admin@demo-cluster.eu-west-2.eksctl.io
```

```
kubectl config delete-cluster demo-cluster.eu-west-2.eksctl.io
```

```
kubectl config delete-user admin@demo-cluster.eu-west-2.eksctl.io
```

Or eksctl may clean it up automatically — check with kubectl config get-contexts after deletion.

Questions

Q:

Can you explain the config yaml file?

A:

The file has three sections:

```
clusters:    ← WHERE to connect (the Kubernetes API server)
contexts:    ← LINKS cluster + user together
users:       ← HOW to authenticate (get a token)
```

clusters section — where to connect and the information to Authenticate to EKS:

```
clusters:
- cluster:
    certificate-authority-data: <certificate-data> ← cluster's public cert
    server: <endpoint-url>                       ← EKS API server address
  name: kubernetes
```

This tells kubectl:

- connect to this EKS API server URL
- trust this certificate (verify you're talking to the right server, not an imposter)

The name: kubernetes is just a label to reference this cluster elsewhere in the file — not the audience (aud).

users section — how to authenticat to AWS:

```
users:
- name: aws
  user:
    exec:
      command: /usr/local/bin/aws-iam-authenticator
      args:
        - "token"
        - "-i"
        - <cluster-name>
```

This tells kubectl:

- when you need a token, run `aws-iam-authenticator token -i <cluster-name>`
- `aws-iam-authenticator` uses your AWS credentials (`~/.aws/credentials`)

- calls AWS STS to get a presigned URL
- returns a token kubectl can present to EKS API server

The name: aws is again just a label — not the aud.

The token argument here is a subcommand of aws-iam-authenticator meaning "give me a token", not the token itself.

contexts section — links the two together:

```
contexts:
- context:
  cluster: kubernetes    ← references clusters[name: kubernetes]
  user: aws              ← references users[name: aws]
  name: aws
current-context: aws
```

This says:

- when using context aws
- connect to cluster named kubernetes
- authenticate using user named aws

The corrected flow:

kubectl get nodes



reads current-context: aws



context links to:

cluster: kubernetes → use this API server URL + certificate

user: aws → run aws-iam-authenticator to get token



kubectl runs: aws-iam-authenticator token -i <cluster-name>



aws-iam-authenticator reads ~/.aws/credentials

calls AWS STS → gets presigned token

returns token to kubectl



kubectl sends request to EKS API server:

GET /api/v1/nodes
Authorization: Bearer <token>
|
▼
EKS API server validates token with AWS IAM
|
▼
checks aws-auth ConfigMap for permissions
|
▼
returns nodes

Q:

when this is executed: aws-iam-authenticator reads ~/.aws/credentials
is this the part where kubectl gets authenticated by AWS? then the next part is
to authenticate with EKS. correct?

A:

Yes exactly — two separate authentication steps:

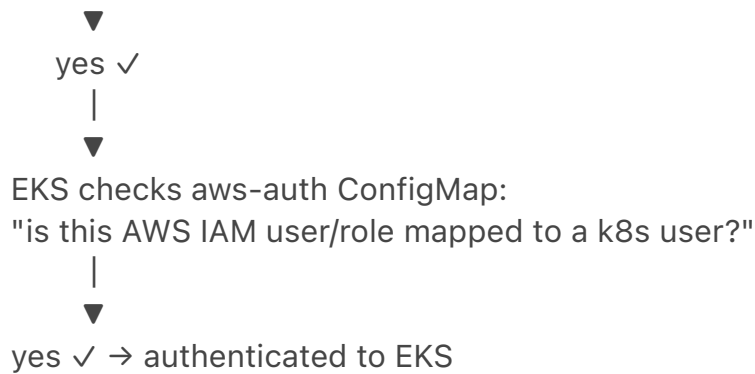
Step 1 — authenticate with AWS:

aws-iam-authenticator reads ~/.aws/credentials
|
▼
uses Access Key ID + Secret Access Key
to call AWS STS
|
▼
AWS STS verifies: are these valid AWS credentials?
|
▼
yes ✓ → AWS STS returns a presigned token

This answers: "are you a valid AWS IAM user?"

Step 2 — authenticate with EKS:

kubectl presents presigned token to EKS API server
|
▼
EKS API server validates token with AWS IAM:
"is this token genuine?"
|



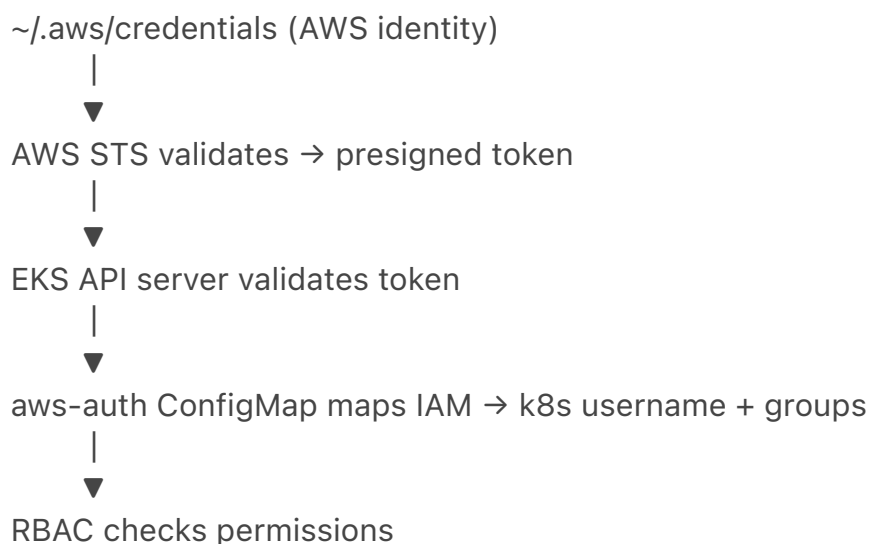
This answers: "is this AWS identity allowed to access this EKS cluster?"

The aws-auth ConfigMap is the bridge:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: aws-auth
  namespace: kube-system
data:
  mapUsers: |
    - userarn: arn:aws:iam::123456789:user/astrid
      username: astrid
    groups:
      - cluster-admins
```

This is the EKS equivalent of the Keycloak user setup we discussed — it maps AWS IAM identities to Kubernetes usernames and groups. Then RBAC takes over from there.

The full chain:





allowed or denied

So yes — two authentications happen sequentially:

1. you → AWS
2. you → EKS (using proof from step 1)

Q:

can you tell me more about aws-auth ConfigMap? what is it? did aws created by default? everytime I have a new user or new role it gets updated automatically? why the Jenkins user was not included in it after I created?

A:

What is aws-auth ConfigMap?

It is a Kubernetes ConfigMap that lives in the kube-system namespace and acts as a bridge between AWS IAM identities and Kubernetes RBAC.

AWS IAM identity (user or role)



aws-auth ConfigMap maps it to



Kubernetes username + groups



RBAC controls what they can do

Without aws-auth, EKS has no way of knowing which AWS IAM identities are allowed to access the cluster.

Did AWS create it by default?

Yes — EKS creates it automatically when you create a cluster. But it only contains the node group role by default:

```
mapRoles:
  - rolearn: arn:aws:iam::706976333940:role/eksctl-demo-cluster-nodegroup-...
    username: system:node:{{EC2PrivateDNSName}}
    groups:
      - system:bootstrappers
      - system:nodes
```

This entry allows your EC2 worker nodes to join the cluster and register themselves. Without it your nodes couldn't communicate with the control plane.

Does it update automatically when you create a new user or role?

No — it never updates automatically. You must manually add every IAM identity that needs cluster access.

create IAM user in AWS

|



AWS has no idea this user needs EKS access

|



you must manually add to aws-auth ConfigMap

|



only then can that user access the cluster

This is by design — AWS separates IAM (who exists) from EKS access (who is allowed in).

Why was Jenkins user not included automatically?

Same reason — you created the Jenkins IAM user in AWS IAM, but AWS has no way of knowing you want that user to access your EKS cluster. There is no automatic link between:

```
IAM user created      ← AWS IAM service
  |
  | no automatic connection
  |
EKS cluster access    ← EKS service (aws-auth ConfigMap)

You have to explicitly make that connection by editing aws-auth.
```

The one exception — the cluster creator:

The IAM user or role that created the cluster gets automatic admin access without being in aws-auth:

your laptop IAM user ran: `eksctl create cluster`

|



that IAM identity gets implicit cluster-admin access

|



no aws-auth entry needed for the creator

That's why you could immediately run `kubectl` from your laptop after creating the cluster — your IAM user was the creator so it had automatic access.

Everyone else — including Jenkins — must be explicitly added to aws-auth.

Summary:

IAM identity	aws-auth entry needed?
Cluster creator	no – automatic
Node group role	no – EKS adds automatically
Any other IAM user	yes – manual
Any other IAM role	yes – manual